

Aircraft-Agnostic Modular Framework(AAMF)

Naman Kumar Shetty

A report submitted in partial fulfillment of the requirements
for the degree of

Master of Science in Aerospace Engineering

Virginia Polytechnic Institute and State University
Blacksburg, Virginia

Committee Chair: Dr. Darshan Sarojini

Committee Members: Dr. Nicholas F. Polys, Dr. Mehdi Vahab

May 2026

Contents

Abstract	1
Nomenclature	2
1 Introduction	3
2 Existing Frameworks and Motivation	6
2.1 Overview of Existing Tools	6
2.1.1 JSBSim	6
2.1.2 FlightGear	7
2.1.3 MADCASP (Auburn University)	7
2.1.4 MADCASP (NASA)	7
2.1.5 X-Plane	8
2.1.6 Emerging Open-Source Simulation Frameworks	8
2.2 Comparative Analysis	9
2.3 Identified Gaps	10
2.4 Motivation for This Work	11
3 Framework Architecture	12
3.1 System Overview	12
3.1.1 Example Fixed-Wing Build-Up	13
3.2 Modeling Assumptions	15
3.3 Coordinate Frames and State Representation	16
3.3.1 Force and Moment Evaluation Pipeline	17
3.4 Aircraft Geometry Module	18
3.5 Aerodynamics Module	19
3.5.1 Coefficient-Based Aerodynamics	19
3.5.2 Wind-Axis Load Aerodynamics	20
3.5.3 Body-Axis Load Aerodynamics	21
3.6 Control Surface Modeling	21

3.7	Control Vector Mapping	22
3.8	Mass Properties Module	22
3.9	Propulsion Module	23
3.10	Multirotor Component Build-Up Using Aggregation Framework	24
3.10.1	Step 1: Point Mass Initialization	24
3.10.2	Step 2: Addition of First Rotor	24
3.10.3	Step 3: Addition of Remaining Rotors	25
3.10.4	Step 4: Mass and Inertia Definition	25
3.10.5	Step 5: Force and Moment Aggregation	25
3.10.6	Failure Scenario	26
3.11	Force and Moment Aggregation	26
3.12	Architecture Summary	27
4	Analysis and Simulation	29
4.1	Trim Analysis	29
4.2	Performance Analysis	31
4.3	Stability Analysis	32
4.4	Mission Planning and Trim Scheduling	33
4.5	Takeoff Analysis	34
4.6	Landing Analysis	35
4.7	Time-Domain Simulation	36
4.8	Generalized Simulink S-Function Interface	37
4.8.1	Rate Limiting	37
4.9	Unified Analysis Pipeline	38
5	Results and Validation	39
5.1	Validation Methodology	39
5.2	Cessna 172 Configuration and Validation	40
5.2.1	Configuration	40
5.2.2	Cruise Performance Comparison	40
5.2.3	Trim Validation	41
5.2.4	Center of Gravity Migration	42
5.3	F-16 Performance Validation and Envelope Analysis	42
5.3.1	Stability Analysis	42
5.4	Mission Planning Validation	43
5.4.1	Rate of Climb as a Function of Velocity	43
5.4.2	Rate of Climb Envelope with Altitude	44

5.4.3	Thrust Available vs Thrust Required	44
5.4.4	Key Observation	45
5.5	Boeing 737 DATCOM Validation	46
5.5.1	Trim and Aerodynamic Performance	46
5.5.2	Takeoff Performance	47
5.5.3	One-Engine-Inoperative (OEI) Climb Performance	47
5.5.4	Landing Performance	48
5.5.5	Stability Analysis	48
5.5.6	Key Observation	49
5.6	Quadrotor Validation	49
5.7	Computational Performance Analysis	49
6	Real-Time Simulation and Visualization	51
6.1	Simulation Hardware Setup	51
6.2	Simulink S-Function and 6-DOF Integration	52
6.3	Real-Time Visualization Pipeline	54
6.4	Decoupled Simulation and Visualization	55
6.5	Platform-Independent Visualization Bridge	55
6.6	Implementation Validation	56
6.7	Summary	56
7	Conclusion and Future Work	58
7.1	Conclusion	58
7.1.1	Limitations of This Work	60
7.2	Future Work	61
7.2.1	Extension to eVTOL and Distributed Propulsion Systems	61
7.2.2	Validation for Conceptual Aircraft Design	62
7.2.3	Integration of High-Fidelity Aerodynamics	62
7.2.4	Enhanced Visualization and Immersive Simulation	62
7.2.5	Generalized Flight Control Framework	62
7.2.6	Multi-Physics and System-Level Integration	62
7.3	Final Remarks	63

Abstract

Traditional flight simulation tools are typically designed for specific aircraft types, limiting their applicability to emerging configurations such as eVTOL aircraft, multirotors, and distributed propulsion systems. Existing frameworks often require separate modeling approaches for different aircraft classes, leading to duplication and inconsistent analysis workflows.

This work addresses these limitations by developing an aircraft-agnostic modular framework for flight simulation and analysis in MATLAB and Simulink. The framework is built around a single **Aircraft** object that aggregates forces and moments from independent subsystems-aerodynamics, propulsion, mass properties, and control without requiring aircraft-specific equations. Propulsion elements contribute moments through $\mathbf{M} = \mathbf{r} \times \mathbf{F}$, enabling natural representation of distributed thrust and asymmetric configurations.

A key design principle is the unified analysis pipeline: the same definition of aircraft is used for trim computation, performance analysis, stability linearization, mission planning, and time-domain simulation. This eliminates model duplication and ensures physical consistency across all analyzes. The trim solver uses nonlinear optimization, stability characteristics are obtained through numerical linearization, and real-time simulation is enabled through a generalized Simulink S-function interface.

The framework decouples simulation and visualization using UDP and WebSocket protocols, allowing integration with external clients such as CesiumJS and Unreal Engine. This architecture supports both software-in-the-loop and hardware-in-the-loop workflows.

Validation was performed using four aircraft configurations: a Cessna 172 (low-speed propeller aircraft), an F-16 (high-performance fighter), a Boeing 737 (DATCOM-based transport model), and a quadrotor (distributed propulsion system). Results demonstrate accurate trim convergence (residuals $< 10^{-3}$), physically consistent stability modes, and performance predictions within 5% of reference data where available.

The framework establishes a unified and extensible approach to aircraft simulation, enabling consistent modeling, analysis, and real-time simulation across both conventional and emerging configurations.

Nomenclature

Symbol	Description
$\mathbf{F}$	Force vector in body frame
$\mathbf{M}$	Moment vector in body frame
$\mathbf{r}$	Position vector from reference point
m	Aircraft mass
$\mathbf{I}$	Inertia tensor
u, v, w	Body-axis velocity components
p, q, r	Body-axis angular rates
ϕ, θ, ψ	Roll, pitch, and yaw angles
V	Airspeed magnitude
ρ	Air density
q_∞	Dynamic pressure ($\frac{1}{2}\rho V^2$)
S	Reference wing area
b	Wing span
$\bar{c}$	Mean aerodynamic chord
C_L, C_D, C_Y	Lift, drag, and side-force coefficients
C_l, C_m, C_n	Roll, pitch, and yaw moment coefficients
α	Angle of attack
β	Sideslip angle
T	Thrust
W	Weight of aircraft
g	Gravitational acceleration
γ	Flight path angle
$\delta_a, \delta_e, \delta_r$	Aileron, elevator, rudder deflections
δ_t	Throttle input
$\mathbf{x}$	State vector
$\mathbf{u}$	Control input vector
I_{xx}, I_{yy}, I_{zz}	Principal moments of inertia

Chapter 1

Introduction

Flight simulation plays a central role in modern aerospace engineering by enabling aircraft behavior to be evaluated prior to physical testing. During the aircraft design process, simulation is used to estimate trim conditions, analyze stability characteristics, evaluate performance metrics, design and test control laws, and study mission-level behavior. These capabilities are essential for reducing development cost, improving safety, and accelerating design iteration [1, 2].

Traditional flight simulation tools are typically developed around a specific aircraft type or a fixed modeling structure. This approach is effective when the vehicle configuration is known in advance, such as a conventional fixed-wing aircraft with predefined aerodynamic coefficients, control surfaces, and propulsion systems. However, this paradigm becomes limiting when the aircraft configuration deviates from conventional designs. Emerging aircraft concepts—including electric vertical takeoff and landing (eVTOL) vehicles, multirotor systems, distributed propulsion aircraft, hybrid-electric configurations, and unconventional fixed-wing designs—require flexible combinations of aerodynamic surfaces, propulsion elements, mass models, and control architectures [17].

A fundamental challenge in developing a generalized aircraft simulation framework is that different aircraft generate forces and moments through fundamentally different physical mechanisms. A conventional fixed-wing aircraft primarily generates lift through wings, control moments through aerodynamic surfaces, and thrust through one or more engines. In contrast, a quadrotor generates both lift and control moments through rotor thrust alone. Hybrid configurations such as eVTOL aircraft combine these mechanisms, using rotors for hover and wings for forward flight. If each aircraft type requires a separate set of governing equations and a dedicated simulation pipeline, the resulting framework becomes difficult to extend, maintain, and generalize.

This work addresses this limitation by developing an aircraft-agnostic modular framework

for flight simulation and analysis using MATLAB and Simulink. The framework is structured around a single **Aircraft** object that serves as the central representation of the vehicle, storing geometry, mass properties, aerodynamic models, propulsion systems, control surfaces, control vectors, and state variables. Rather than hardcoding equations for specific aircraft configurations, each subsystem independently computes its contribution to the total force and moment acting on the aircraft. These contributions are then transformed into a common body-axis representation and aggregated about a consistent reference point.

The core concept underlying this framework is the force and moment aggregation approach. Aerodynamic forces, propulsive forces, gravitational effects, and additional component-level contributions are computed independently and combined to obtain the total forces and moments acting on the aircraft. This formulation is expressed as:

The total force and moment acting on the aircraft, expressed in the body-fixed frame and evaluated about the reference point (typically the center of gravity), are given by:

$$\mathbf{F}_{\text{total}}^b = \mathbf{F}_{\text{aero}}^b + \mathbf{F}_{\text{prop}}^b + \mathbf{F}_{\text{gravity}}^b \quad (1.1)$$

$$\mathbf{M}_{\text{total}}^b = \mathbf{M}_{\text{aero}}^b + \mathbf{M}_{\text{prop}}^b + \mathbf{M}_{\text{gravity}}^b \quad (1.2)$$

In addition, propulsion elements may generate moments due to their spatial offset from the reference point. This contribution is modeled using the standard rigid-body moment relationship:

$$\mathbf{M}_{\text{prop}} = \mathbf{r} \times \mathbf{F}_{\text{prop}} + \mathbf{M}_{\text{extra}} \quad (1.3)$$

This formulation is consistent with classical rigid-body dynamics and enables accurate representation of distributed propulsion systems, asymmetric thrust conditions, and rotor-based configurations [4]. The same mathematical structure applies regardless of aircraft type, providing a unified representation for fixed-wing aircraft, rotorcraft, and hybrid configurations.

The formulation introduced in this chapter is further developed in Chapter 3, where the aggregation framework is presented in detail along with subsystem-level modeling and implementation.

The objectives of this work are summarized as follows:

- Develop a modular and extensible framework for aircraft modeling
- Enable unified analysis and simulation using a single aircraft representation

- Support multiple aircraft configurations without modifying the core structure
- Integrate real-time simulation and visualization capabilities

By achieving these objectives, this work aims to provide a flexible and scalable simulation platform suitable for both conceptual aircraft design and real-time simulation applications. In particular, the framework is designed to bridge the gap between traditional flight dynamics analysis tools and modern multi-configuration simulation environments, enabling consistent modeling across emerging aerospace systems.

Chapter 2

Existing Frameworks and Motivation

Existing flight simulation tools have reached a high level of maturity in the context of fixed-wing aircraft, offering validated 6-DOF dynamics, photorealistic visualization, and hardware-in-the-loop (HIL) interfaces. These frameworks have been successfully applied in both academic research and industrial applications for modeling conventional aircraft dynamics [1, 4].

However, the emergence of novel aircraft configurations , including electric vertical take-off and landing (eVTOL) systems, multicopter vehicles, distributed propulsion aircraft, and hybrid-propulsion designs — has exposed fundamental limitations in the flexibility and extensibility of these frameworks. Such configurations involve strong coupling between aerodynamics and propulsion, as well as nontraditional force generation mechanisms that are not easily represented within conventional simulation architectures [17].

This chapter reviews widely used simulation environments, evaluates their capabilities and limitations, and establishes the motivation for developing an aircraft-agnostic modular framework.

2.1 Overview of Existing Tools

2.1.1 JSBSim

JSBSim is a widely used open-source flight dynamics model written in C++ that provides a validated 6-DOF simulation engine [11]. Aircraft are defined through XML configuration files specifying aerodynamic coefficients, mass properties, and engine parameters. The framework supports fuel and energy modeling and is widely used in both research and industry.

However, propulsion modeling in JSBSim is primarily based on aggregated aerodynamic coefficients rather than explicit per-engine force and moment contributions resolved at individual engine locations. This limits its ability to accurately represent distributed propulsion

systems or asymmetric thrust effects.

From a structural perspective, the propulsion model does not naturally capture moment contributions of the form $\mathbf{r} \times \mathbf{F}$ at the component level, which becomes critical in configurations with multiple propulsors. Visualization is typically achieved through integration with FlightGear, creating a dependency that reduces architectural flexibility. Furthermore, JSB-Sim does not natively support unified modeling of multi-modal aircraft combining fixed-wing and multirotor dynamics within a single consistent formulation.

2.1.2 FlightGear

FlightGear is a full-featured open-source flight simulator that provides built-in 3D visualization and HIL interfaces. It uses JSBSim as its underlying flight dynamics engine and therefore inherits similar modeling constraints [12].

As a simulation platform rather than a modeling framework, FlightGear is not designed for extensibility or integration into analysis workflows such as trim computation, stability analysis, or mission planning. Its architecture is primarily oriented toward visualization and user interaction, rather than flexible physics modeling.

2.1.3 MADCASP (Auburn University)

The Multi-Aircraft Dynamics, Control, and Simulation Platform (MADCASP) developed at Auburn University provides a Simulink-based environment for flight simulation [13]. It supports certain multi-modal configurations such as tiltrotor transition modeling.

However, it does not natively support pure quadrotor or general multirotor configurations. The framework relies heavily on predefined Simulink block structures and integrates with FlightGear for visualization, limiting flexibility. Additionally, its academic licensing model restricts accessibility and broader adoption.

2.1.4 MADCASP (NASA)

The NASA implementation of MADCASP is developed in C++ and Fortran and targets high-fidelity aerospace analysis workflows. While it supports detailed modeling capabilities, propulsion is typically modeled as total thrust, with moment contributions often computed externally rather than as part of an integrated framework.

Visualization is handled through RT3D, and the framework is not publicly available. This limits its use in academic research and prevents extensibility for new aircraft configurations.

2.1.5 X-Plane

X-Plane is a commercial flight simulator that employs blade element theory for aerodynamic modeling [14]. This approach provides improved physical fidelity compared to purely coefficient-based models by resolving forces at the element level.

However, its closed-source architecture significantly limits its applicability as a research platform. The lack of programmatic interfaces and restricted extensibility makes it unsuitable for developing new aircraft configurations, integrating custom propulsion models, or performing advanced analysis workflows such as optimization, stability linearization, or mission planning.

2.1.6 Emerging Open-Source Simulation Frameworks

Recent developments in open-source simulation environments have explored modular and multi-domain simulation approaches, particularly in the context of autonomy, system integration, and urban air mobility. These frameworks emphasize flexibility and interoperability, but differ significantly in their treatment of aircraft physics and modeling fidelity.

OpenSim

OpenSim is a browser-based simulation framework that enables real-time aircraft and mission simulation through JSON-defined configurations [18]. The framework is designed for accessibility and modularity, allowing rapid prototyping of scenarios without requiring compilation or complex setup.

The underlying physics model is based on simplified aerodynamic force balance equations, where lift, drag, thrust, and weight are computed using classical relations. However, the system employs a point-mass formulation with approximated angular dynamics rather than a full rigid-body representation. Propulsion is modeled as a scalar thrust input, without explicit spatial force application or moment generation.

While OpenSim supports multi-domain scenarios such as aircraft, rockets, and orbital motion, these are implemented as separate physics modules rather than through a unified formulation. As a result, the framework is well suited for visualization and training applications, but does not provide the level of fidelity required for flight dynamics analysis, control design, or distributed propulsion modeling.

AeroSim

AeroSim is a modern, scalable simulation framework designed for system-level integration of aerospace systems [19, 20]. The architecture follows a modular design, where dynamics models, controllers, sensors, and environment components are implemented as independent subsystems and connected through standardized interfaces.

A key feature of AeroSim is its support for the Functional Mock-up Interface (FMI), allowing components to be encapsulated as Functional Mock-up Units (FMUs). These components communicate through middleware layers, enabling co-simulation with tools such as Simulink and integration with visualization platforms including Unreal Engine and Omniverse.

However, AeroSim does not enforce a unified physics formulation at the aircraft level. Instead, flight dynamics models are imported as external FMUs, and the framework does not prescribe how forces and moments are computed or aggregated. Consequently, physical consistency depends on the specific models integrated into the system. AeroSim is therefore best characterized as a system-level simulation and integration platform rather than a physics-based aircraft modeling framework.

UAM-SUMO

UAM-SUMO extends the Simulation of Urban Mobility (SUMO) platform to study the large-scale impact of urban air mobility systems [21]. The framework introduces air taxi operations into urban traffic simulations, enabling analysis of mode choice, traffic flow, and system-level performance.

The system models air taxis as transport agents characterized by parameters such as capacity, speed, and routing behavior. Inter modal routing is supported through integration of pedestrian, ground, and aerial transport modes, allowing evaluation of travel efficiency and congestion effects in urban environments.

However, UAM-SUMO operates at a high level of abstraction and does not model aircraft physics. Aerodynamic forces, propulsion systems, and flight dynamics are not explicitly represented, and aircraft are represented as transportation agents within the network model. As a result, the framework is not suitable for aircraft-level analysis, but is effective for studying large-scale system behavior and infrastructure planning.

2.2 Comparative Analysis

Table 2.1 summarizes key characteristics of existing frameworks.

Table 2.1: Comparison of existing flight simulation frameworks

Framework	Multi-modal	Energy Model	Propulsion	Architecture	Visualization	Open Source
JSBSim	✗	✓	Total Coeff.	XML Config	FlightGear	✓
FlightGear	✗	Via JSBSim	Via JSBSim	Application	Built-in 3D	✓
MADCASP (Auburn)	Partial ^a	✓	Simulink Blocks	Simulink Model	FlightGear	Academic
MADCASP (NASA)	✗	✓	Total Thrust ^b	C++/Fortran (OOP)	RT3D	✗
X-Plane	✗	✗	Blade Element	Closed	Built-in 3D	✗
OpenSim	✓	✗	Scalar Thrust	JSON / Browser-Based	Web-Based	✓
AeroSim	✓	Partial	External FMU Models	Distributed Modular	Unreal / Omniverse	✓
UAM-SUMO	✓	✗	Abstract Agent Model	Traffic Simulation	SUMO GUI	✓
This Work	✓	✓	Per-Engine ($\mathbf{r} \times \mathbf{F}$)	Abstract OOP	CesiumJS / Unreal / FlightGear	✓

^a Supports tiltrotor but not general multirotor configurations.

^b Moment contributions often require manual computation.

2.3 Identified Gaps

Several limitations are observed across existing flight simulation frameworks:

Lack of unified multi-modal support. Most frameworks do not support both fixed-wing and multirotor configurations within a single architecture, requiring separate modeling approaches. This becomes a significant limitation for hybrid systems such as eVTOL aircraft, where multiple flight regimes must be represented within the same model.

Simplified propulsion modeling. Propulsion is often represented as total thrust, without accounting for individual force and moment contributions from each propulsion unit. This limits fidelity when modeling distributed propulsion systems, particularly under asymmetric or failure conditions.

Tightly coupled visualization. Many frameworks rely on integrated visualization environments, which reduces flexibility when interfacing with modern rendering platforms such as web-based tools or game engines.

Limited software extensibility. Rigid architectures make it difficult to extend models to new aircraft configurations without modifying core simulation code. This increases development effort and limits scalability across different vehicle types.

Separation of analysis and simulation. Trim, performance analysis, stability analysis, and time-domain simulation are often implemented as separate workflows. This can lead to inconsistencies and duplication of models across different analyses.

Lack of unified physics-level modeling in modern frameworks. Recent open-source frameworks emphasize modularity and system integration, but often operate either at a high level of abstraction (e.g., UAM-SUMO) or rely on externally defined models for physical behavior (e.g., AeroSim). As a result, they do not enforce a consistent force and moment formulation across different aircraft configurations.

These limitations arise largely from how the frameworks are architected. As aircraft concepts evolve toward hybrid configurations, distributed propulsion, and multi-regime operation, these design limitations make it difficult for existing frameworks to scale effectively.

2.4 Motivation for This Work

The limitations identified above motivate the development of an aircraft-agnostic modular framework using MATLAB and Simulink.

The key design principles are:

- **Single Aircraft object:** A unified representation of geometry, aerodynamics, propulsion, and mass properties, ensuring consistency across all analyses.
- **Per-engine force and moment modeling:** Each propulsion unit contributes independently:

$$\mathbf{M}_i = \mathbf{r}_i \times \mathbf{F}_i \quad (2.1)$$

This enables accurate modeling of distributed propulsion, rotorcraft, and asymmetric thrust conditions.

- **Object-oriented modular design:** New subsystem models can be introduced without modifying the aircraft-level equations of motion.
- **Decoupled visualization:** Simulation is independent of rendering platforms, supporting CesiumJS, Unreal Engine, and other visualization tools.
- **Unified pipeline:** A single aircraft definition is shared across trim analysis, performance evaluation, stability analysis, and time-domain simulation, ensuring physical consistency.

These principles directly address the limitations identified in existing frameworks and form the foundation of the aircraft-agnostic simulation architecture developed in this work.

Chapter 3

Framework Architecture

3.1 System Overview

The proposed aircraft-agnostic modular framework is built around a central **Aircraft** object, which serves as the single source of truth for the entire vehicle definition. Instead of creating separate models for trim, performance, stability, simulation, and visualization, a single aircraft definition is shared across all analysis and simulation modules. This design avoids inconsistencies that can arise when separate models are maintained for different analyses.

The **Aircraft** object contains references to all major physical subsystems, including geometry, mass properties, aerodynamics, control surfaces, propulsion elements, state representation, and control vector mapping. Analysis modules such as trim, performance, stability, takeoff, landing, and mission planning operate directly on this shared aircraft definition. This eliminates duplication of data and ensures consistency across all computations.

The framework follows a modular object-oriented design. Each discipline model is implemented as an independent class with a common interface, allowing aerodynamic, propulsion, and mass models to be interchanged without modifying the aircraft-level simulation structure. This abstraction is consistent with standard simulation architecture practices, where model components are separated from the numerical simulation [1].

A key motivation for this architecture is reusability. In traditional aircraft simulation workflows, changes to the aircraft configuration often require changes to the analysis model, simulation model, and visualization interface. In the proposed framework, the physical configuration is modified by changing the components attached to the **Aircraft** object, while

the analysis and simulation pipeline remains unchanged. This makes the framework suitable for comparing fundamentally different aircraft configurations within a common software structure.

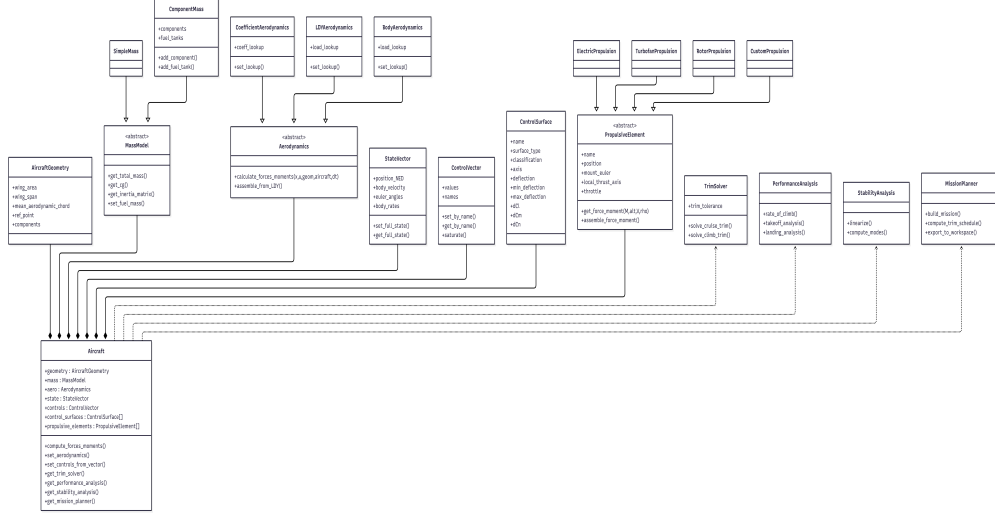


Figure 3.1: UML class structure of the aircraft-agnostic modular framework.

The UML diagram in Fig. 3.1 illustrates the hierarchical organization of the framework. The **Aircraft** object aggregates subsystem classes such as geometry, mass, aerodynamics, propulsion, and control. Each subsystem is abstracted through a common interface, allowing the aircraft object to request forces, moments, and other outputs without requiring knowledge of the internal implementation of each model.

This abstraction is central to the aircraft-agnostic nature of the framework. The aircraft-level model does not need to know whether aerodynamic data comes from coefficient lookup tables, wind-axis loads, body-axis loads, or a future high-fidelity solver. Similarly, the aircraft-level model does not need to know whether thrust is generated by a piston propeller, turbofan, electric motor, rotor, or custom user-defined propulsion element. Each subsystem is responsible for computing its own contribution, and the aircraft object is responsible for combining those contributions consistently.

3.1.1 Example Fixed-Wing Build-Up

Figure 3.2 illustrates the construction of a fixed-wing aircraft model using the proposed modular framework. Rather than defining a complete aircraft through a monolithic set of equations, the framework incrementally builds the vehicle by assembling independent physical subsystems into a unified representation.

The build-up process begins with the initialization of an empty **Aircraft** object, which acts as the central container for all aircraft-related data. At this stage, no physical properties

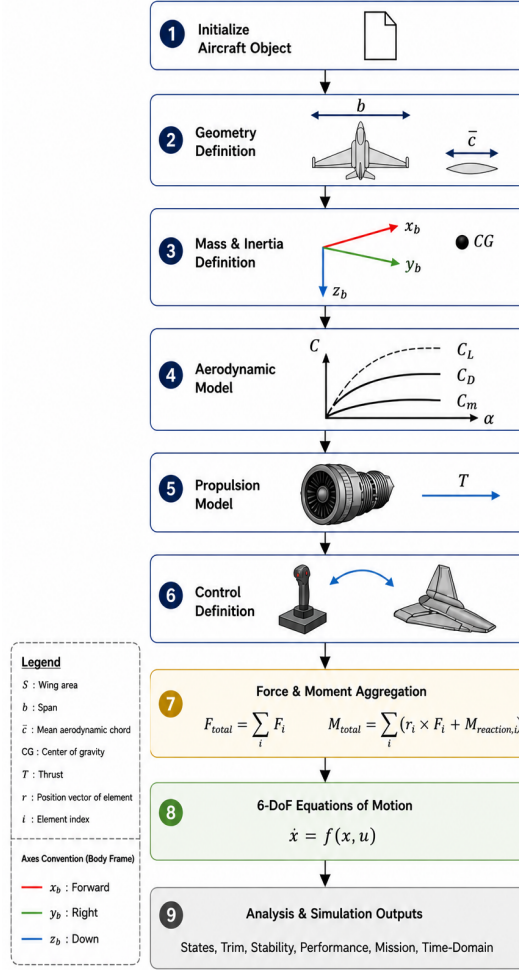


Figure 3.2: Process-oriented fixed-wing aircraft build-up using the modular framework. Individual subsystems are incrementally added and combined into a single aircraft representation.

or dynamics are defined.

In the first step, the geometry module is introduced. This defines the fundamental reference parameters of the aircraft, including wing area S , span b , and mean aerodynamic chord $\bar{c}$. These quantities are essential for aerodynamic force and moment calculations and establish the geometric scaling of the system.

Next, the mass properties are defined. The mass module specifies the total mass, center of gravity (CG) location, and inertia tensor. These properties directly influence the translational and rotational dynamics of the aircraft and are used by the 6-DOF equations of motion.

Following this, the aerodynamic model is added. Depending on the fidelity level, this may consist of coefficient-based models (e.g., C_L , C_D , C_m) or lookup tables derived from empirical methods such as DATCOM [30]. The aerodynamic module computes forces and

moments based on flight condition variables such as angle of attack, sideslip, and control surface deflections.

The propulsion system is then incorporated through one or more `PropulsiveElement` objects. Each propulsion element generates a force vector along its thrust direction and contributes a moment based on its offset from the reference point, as defined in Eq. (3.21). This formulation allows both single-engine and multi-engine configurations to be modeled consistently.

Finally, the control system is defined. Control surfaces such as ailerons, elevator, and rudder are introduced, along with throttle inputs. These are mapped into a unified control vector, which serves as the input to the aircraft model. Control inputs modify aerodynamic coefficients and propulsion outputs, enabling maneuvering and trim capability.

Once all subsystems are defined, the aircraft model is assembled into a single representation:

$$\mathcal{A} = \{\text{Geometry, Mass, Aerodynamics, Propulsion, Control}\} \quad (3.1)$$

At this point, the aircraft object is fully defined and capable of producing total force and moment outputs using the aggregation formulation introduced in Eqs. (3.36)–(3.37).

A key advantage of this approach is that the same aircraft definition is reused across all analysis modules. The constructed aircraft object can be directly passed to trim solvers, stability analysis routines, performance evaluation tools, mission planners, and time-domain simulations without modification or duplication.

This example reflects a conventional fixed-wing modeling workflow; however, unlike traditional frameworks, the underlying equations are not specific to this configuration. The same assembly process and aggregation formulation apply to fundamentally different aircraft types, enabling a unified and extensible simulation architecture.

3.2 Modeling Assumptions

The formulation of the proposed framework is based on the following modeling assumptions:

- **Rigid-body dynamics:** The aircraft is modeled as a rigid body. Structural flexibility and aeroelastic effects are neglected.
- **Six-degree-of-freedom motion:** The aircraft motion is represented using a 12-state formulation, including translational and rotational dynamics in the body frame.
- **Body-axis force and moment representation:** All forces and moments are resolved in the body-fixed coordinate system before aggregation.

- **Consistent reference point:** Forces and moments are evaluated about a defined reference point, typically the center of gravity, unless otherwise specified.
- **Standard atmosphere:** Atmospheric properties are computed using the International Standard Atmosphere (ISA) model. Effects such as turbulence, wind, and temperature variations are not included.
- **Quasi-steady aerodynamics:** Aerodynamic forces and moments are assumed to depend instantaneously on the current state and control inputs. Unsteady aerodynamic effects are not modeled.
- **Simplified propulsion dynamics:** Propulsion systems generate thrust based on current inputs without modeling internal engine transients. However, the framework supports integration of higher-fidelity propulsion models.
- **Direct control inputs:** Control inputs are applied directly to the system without explicit modeling of actuator dynamics or feedback control systems.
- **Flat-Earth approximation:** The Earth is assumed to be flat over the scale of motion considered, and curvature effects are neglected.

3.3 Coordinate Frames and State Representation

The framework uses a North-East-Down (NED) inertial frame and a body-fixed coordinate frame. The NED frame is used for vehicle position, trajectory, and navigation, while the body frame is used for forces, moments, velocities, and angular rates. This convention is commonly used in flight dynamics and inertial navigation formulations [1, 2].

The body-axis convention is defined as:

- x_b : positive forward along the aircraft nose,
- y_b : positive toward the right wing,
- z_b : positive downward.

Unless otherwise specified, all forces and moments used in the framework are expressed in the body-fixed coordinate system and evaluated about a consistent reference point, typically the center of gravity. Any quantities defined in alternate frames (e.g., wind or inertial frames) are transformed into the body frame prior to aggregation.

Maintaining a consistent axis convention is essential because aerodynamic, propulsive, gravitational, and inertial quantities may initially be defined in different frames. For example, aerodynamic lift and drag are often defined in wind axes, thrust may be defined in a local engine frame, and gravity is naturally defined in an inertial frame. Before forces and moments can be summed, all contributions must be transformed into a common reference frame.

The aircraft state is represented using a 12-state rigid-body formulation:

$$6 \begin{bmatrix} p_N & p_E & p_D & u & v & w & \phi & \theta & \psi & p \\ q & r & & & & & & & & \end{bmatrix}^T \quad (3.2)$$

where p_N , p_E , and p_D are the NED position components; u , v , and w are body-axis velocity components; ϕ , θ , and ψ are Euler angles; and p , q , and r are body-axis angular rates. This representation follows the standard rigid-body aircraft dynamics formulation used in classical flight dynamics literature [1, 4, 3].

$$C_b^n = \begin{bmatrix} c\theta c\psi & c\theta s\psi & -s\theta \\ s\phi s\theta c\psi - c\phi s\psi & s\phi s\theta s\psi + c\phi c\psi & s\phi c\theta \\ c\phi s\theta c\psi + s\phi s\psi & c\phi s\theta s\psi - s\phi c\psi & c\phi c\theta \end{bmatrix} \quad (3.3)$$

Equation (3.3) defines the transformation from the body frame to the NED frame. The framework uses the inverse of this transformation to express inertial quantities, such as gravity, in the body frame before force and moment aggregation.

The `StateVector` class manages this representation and provides a consistent interface for accessing position, velocity, attitude, altitude, airspeed, and angular rates. This ensures that all framework modules use the same state convention.

3.3.1 Force and Moment Evaluation Pipeline

Figure 3.3 shows the force and moment evaluation pipeline used throughout the framework.

Aerodynamic loads may initially be defined in wind axes, propulsion loads may be defined along local engine or rotor axes, and gravity is naturally computed in the inertial frame. Before aggregation, all forces and moments are expressed in the body frame. The final output of the framework is the total body-axis force vector and total body-axis moment vector:

$$\mathbf{F}_{body} = \begin{bmatrix} F_x & F_y & F_z \end{bmatrix}^T, \quad \mathbf{M}_{body} = \begin{bmatrix} M_x & M_y & M_z \end{bmatrix}^T. \quad (3.4)$$

Equation (3.4) defines the total body-axis force and moment acting on the aircraft. The framework evaluates these quantities about a consistent reference point after transforming

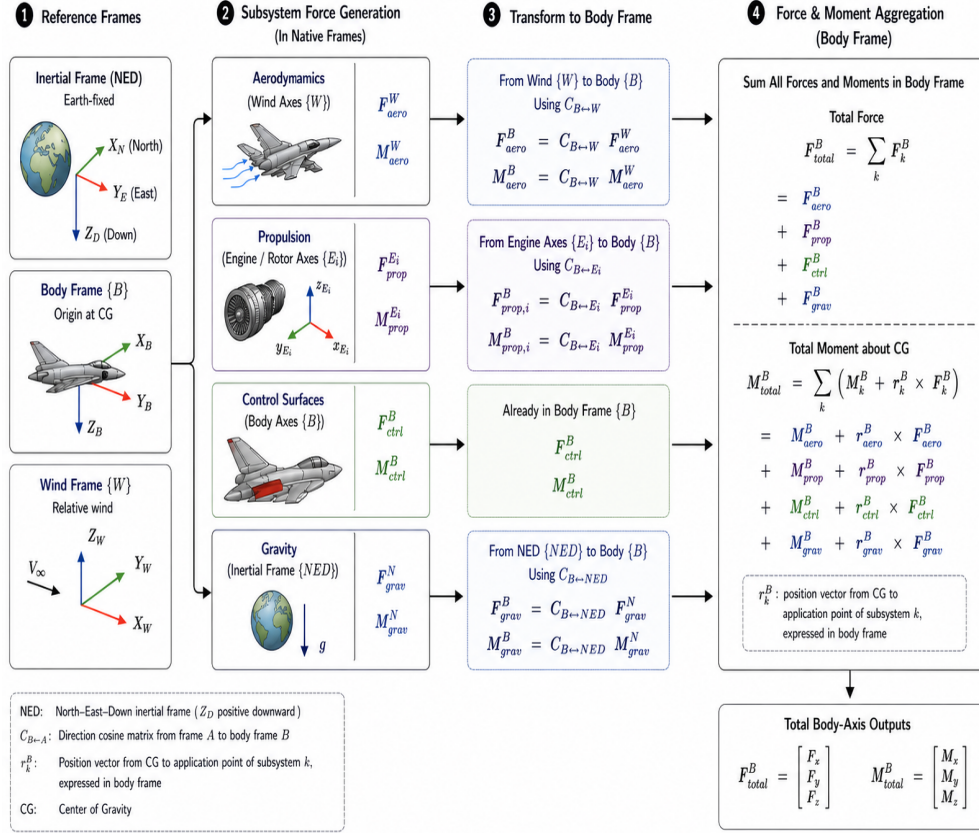


Figure 3.3: Reference frames, subsystem force generation, force and moment aggregation, and 6-DOF dynamics pipeline.

and aggregating contributions from all subsystems.

This formulation ensures physical consistency when combining aerodynamic, propulsion, and gravitational effects. The framework then supplies these total force and moment vectors to the 6-DOF rigid-body equations of motion described in Chapter 4.

The purpose of this pipeline is to ensure that every subsystem, regardless of its internal model type, produces outputs compatible with the same aircraft-level dynamics model. This is what allows a Cessna 172, an F-16, a Boeing 737, and a quadrotor to be represented using the same simulation structure.

3.4 Aircraft Geometry Module

The `AircraftGeometry` class stores the reference geometric quantities required for aerodynamic and performance calculations. These include wing reference area S , span b , mean aerodynamic chord $\bar{c}$, and reference point location.

These parameters are used to dimensionalize aerodynamic coefficients:

$$L = C_L q_\infty S, \quad D = C_D q_\infty S \quad (3.5)$$

$$M_y = C_m q_\infty S \bar{c}, \quad M_x = C_l q_\infty S b, \quad M_z = C_n q_\infty S b \quad (3.6)$$

where L is lift, D is drag, and M_x , M_y , and M_z are roll, pitch, and yaw moments. This coefficient-to-load formulation follows classical aerodynamic modeling practice and is consistent with DATCOM [30]-style aerodynamic data usage

The geometry module supports both simple parameter-based definitions and more detailed component-based descriptions. In early design use cases, only reference quantities may be required. For more detailed configurations, the same class can store named components and reference locations, enabling more structured aircraft definitions.

This is useful because not all aircraft require the same level of geometric detail. A conventional aircraft may require wing area, span, and mean aerodynamic chord for coefficient-based aerodynamic models. A multirotor may not rely on wing reference quantities for lift generation, but still benefits from storing reference dimensions, rotor arm locations, and component positions. Therefore, the geometry module acts as a common container rather than a fixed aircraft-shape assumption.

3.5 Aerodynamics Module

Aerodynamics are implemented through an abstract interface, allowing multiple aerodynamic modeling approaches to be used interchangeably. This is important because aerodynamic data may come from different sources depending on the stage of design. Early-stage models may rely on analytical estimates or DATCOM data [30], while later-stage models may use lookup tables, vortex-lattice methods, wind tunnel data, or CFD-generated loads.

The abstraction also separates aerodynamic data representation from the aircraft-level force and moment aggregation. As long as the aerodynamic model returns a force and moment contribution in the required format, the aircraft object can use it without modification.

3.5.1 Coefficient-Based Aerodynamics

This formulation assumes quasi-steady aerodynamic behavior. The `CoefficientAerodynamics` model uses nondimensional coefficients:

$$C_L, C_D, C_Y, C_l, C_m, C_n \quad (3.7)$$

These are converted into dimensional loads using dynamic pressure:

$$q_\infty = \frac{1}{2}\rho V^2 \quad (3.8)$$

where ρ is air density and V is the total airspeed. This formulation is standard in aircraft performance and stability analysis [1, 30].

The model also supports optional augmentation using static and dynamic stability derivatives. For example, the pitching moment coefficient can be written as:

$$C_m = C_{m,base} + C_{m_\alpha}\alpha + C_{m_q}\hat{q} \quad (3.9)$$

where α is angle of attack and $\hat{q}$ is the nondimensional reduced pitch rate:

$$\hat{q} = \frac{q\bar{c}}{2V} \quad (3.10)$$

This allows the same aerodynamic class to represent simple static coefficient models or more complete dynamic derivative models. For low-fidelity conceptual studies, the coefficient lookup may return only base coefficients. For higher-fidelity stability analysis, the same interface can include derivative terms such as C_{m_α} , C_{m_q} , C_{l_p} , C_{n_r} , and other damping derivatives.

3.5.2 Wind-Axis Load Aerodynamics

The LDYAerodynamics model accepts dimensional wind-axis loads:

$$L, D, Y, M_x, M_y, M_z \quad (3.11)$$

where L , D , and Y represent lift, drag, and side force. These loads are transformed into the body frame before aggregation. This model is useful when aerodynamic data is already available as dimensional loads rather than nondimensional coefficients. In the wind-axis formulation, aerodynamic forces are initially defined in the wind frame as lift (L), drag (D), and side force (Y). These forces must be transformed into the body frame before aggregation.

The framework transforms wind-axis forces into body-axis forces before aggregation. The transformation is given by:

$$\begin{bmatrix} F_x \\ F_y \\ F_z \end{bmatrix} = \begin{bmatrix} -\cos\alpha\cos\beta & \sin\beta & -\sin\alpha\cos\beta \\ -\cos\alpha\sin\beta & -\cos\beta & -\sin\alpha\sin\beta \\ \sin\alpha & 0 & -\cos\alpha \end{bmatrix} \begin{bmatrix} D \\ Y \\ L \end{bmatrix} \quad (3.12)$$

Equation (3.12) ensures that aerodynamic loads defined in wind axes remain consistent

with the body-axis force aggregation used throughout the framework. Here, α represents the angle of attack and β represents the sideslip angle.

The framework uses the transformed forces in the aggregation step to ensure that all subsystem contributions are expressed in a common coordinate system before aggregation. where α is the angle of attack and β is the sideslip angle. This approach can be useful when aerodynamic loads are obtained from external calculations or lookup tables that directly provide lift, drag, and side force. It allows the user to bypass coefficient dimensionalization while still preserving the same aircraft-level aggregation structure.

3.5.3 Body-Axis Load Aerodynamics

The `BodyAerodynamics` model directly accepts body-axis forces and moments:

$$F_x, F_y, F_z, M_x, M_y, M_z \quad (3.13)$$

This approach is useful when aerodynamic data is obtained from high-fidelity solvers or external tools that already output forces and moments in the body frame. In this case, no wind-axis-to-body-axis transformation is required.

By supporting coefficient-based, wind-axis load, and body-axis load models, the framework can operate with aerodynamic data at different fidelity levels without requiring changes to the aircraft-level simulation logic.

3.6 Control Surface Modeling

Control surfaces are represented using the `ControlSurface` class, which defines deflection limits, current deflection, and incremental aerodynamic effects. Each surface is assigned an axis mask:

$$\mathbf{a}_{cs} = [a_l \ a_m \ a_n] \quad (3.14)$$

where a_l , a_m , and a_n indicate whether the surface contributes to roll, pitch, or yaw moment channels.

This enables unified modeling of different control types:

$$\text{Aileron} = [1 \ 0 \ 0], \quad \text{Elevator} = [0 \ 1 \ 0], \quad \text{Rudder} = [0 \ 0 \ 1]. \quad (3.15)$$

Control increments modify aerodynamic coefficients. For example, roll moment coefficient augmentation can be represented as:

$$C_l = C_{l_i} + \Delta C_{l_i} \delta a_{l_i} \quad (3.16)$$

where δ is the control deflection. Similar terms may be used for pitching and yawing moment increments. This formulation allows conventional and mixed-control surfaces such as elevons to be represented through the same structure.

For example, an elevator primarily affects pitching moment and may be assigned an axis mask $[0 \ 1 \ 0]$, while an elevon can affect both roll and pitch and may be assigned a mixed axis mask. This provides a compact way to represent conventional and unconventional control surfaces without creating separate equations for each surface type.

3.7 Control Vector Mapping

The `ControlVector` class assembles all control inputs into a single vector. For a conventional fixed-wing aircraft, the vector may be written as:

$$\mathbf{u}_{fixed} = [\delta_a \ \delta_e \ \delta_r \ \delta_t]^T \quad (3.17)$$

where δ_a , δ_e , δ_r , and δ_t represent aileron, elevator, rudder, and throttle commands.

For multicopter systems, the control vector may instead represent rotor thrust commands:

$$\mathbf{u}_{multi} = [T_1 \ T_2 \ T_3 \ T_4]^T. \quad (3.18)$$

This abstraction is one of the mechanisms that makes the framework aircraft-agnostic. The solver and simulation pipeline do not require a fixed control layout; they only require a control vector consistent with the components registered in the aircraft object.

The same trim solver can therefore operate on different aircraft types by changing the size and interpretation of the control vector. For a fixed-wing aircraft, the solver may vary elevator and throttle. For a quadrotor, it may vary individual rotor thrust commands. The analysis algorithm remains unchanged, while the aircraft definition determines how controls affect forces and moments.

3.8 Mass Properties Module

Mass properties are defined through an abstract interface with implementations such as `SimpleMass` and `ComponentMass`. The `SimpleMass` model is suitable for early conceptual modeling where total mass, center of gravity, and inertia are known directly. The

ComponentMass model is suitable when mass properties are assembled from individual components.

The center of gravity is computed as:

$$\mathbf{r}_{CG} = \frac{\sum_i m_i \mathbf{r}_i}{\sum_i m_i} \quad (3.19)$$

The inertia tensor is assembled using the parallel-axis theorem:

$$\mathbf{I}_{total} = \sum_i [\mathbf{I}_{i,local} + m_i(\|\mathbf{d}_i\|^2 \mathbf{I}_3 - \mathbf{d}_i \mathbf{d}_i^T)] \quad (3.20)$$

where $\mathbf{d}_i$ is the vector from the total aircraft center of gravity to the component center of gravity. This formulation allows mass, center of gravity, and inertia to change as fuel is consumed or component masses are updated.

This capability is important for aircraft where fuel burn or payload changes affect trim and stability. As fuel mass changes, the center of gravity and inertia may shift, causing changes in required control deflection, stability derivatives, and dynamic response. By treating mass as a module, the framework can update these properties consistently during analysis or simulation.

3.9 Propulsion Module

Propulsion is modeled using the abstract **PropulsiveElement** class. Each propulsion unit stores its position, mounting orientation, thrust direction, and throttle command. Each element contributes independently to the total force and moment.

For a propulsion element producing force $\mathbf{F}_i$ at position $\mathbf{r}_i$, the moment contribution is:

$$\mathbf{M}_i = \mathbf{r}_i \times \mathbf{F}_i + \mathbf{M}_{extra} \quad (3.21)$$

where $\mathbf{M}_{extra}$ may include effects such as rotor reaction torque or user-defined additional moments. This formulation, also introduced in Chapter 1 using Eq. (1.3), enables distributed propulsion systems and asymmetric thrust conditions to be modeled naturally.

The propulsion abstraction supports turbofan, propeller, electric, rotor, and custom propulsion models. For example, a conventional aircraft may use a single propeller or turbofan element, while a multirotor may use several rotor elements located at different positions.

This component-level propulsion treatment is especially important for distributed propulsion and multirotor systems. In these configurations, the moment generated by thrust location may be as important as the thrust magnitude itself. Modeling each propulsor indepen-

dently allows the framework to capture asymmetric thrust, rotor failure cases, and off-axis thrust effects without requiring special aircraft-specific equations.

3.10 Multirotor Component Build-Up Using Aggregation Framework

To demonstrate the aircraft-agnostic nature of the proposed framework, a quadrotor configuration is constructed using the same modular architecture. Unlike conventional aircraft, where lift and control are generated through aerodynamic surfaces, a multirotor generates both forces and moments through distributed propulsion elements.

3.10.1 Step 1: Point Mass Initialization

The system begins with a point mass representation at the center of gravity (CG). The state vector is defined using the standard 12-state formulation introduced earlier:

$$\mathbf{x} = \begin{bmatrix} p_N & p_E & p_D & u & v & w & \phi & \theta & \psi & p \\ q & r & & & & & & & & \end{bmatrix}^T \quad (3.22)$$

At this stage, no forces or moments are applied, and the control vector is empty:

$$\mathbf{u} = [\] \quad (3.23)$$

3.10.2 Step 2: Addition of First Rotor

A propulsion element is added at a position $\mathbf{r}_1$ relative to the CG. The rotor generates thrust along the body-axis direction:

$$\mathbf{F}_1 = k_T T_1 \mathbf{z}_b \quad (3.24)$$

The corresponding moment contribution is:

$$\mathbf{M}_1 = \mathbf{r}_1 \times \mathbf{F}_1 + \mathbf{M}_{reaction} \quad (3.25)$$

For rotary propulsion elements, the reaction torque is modeled as:

$$\mathbf{M}_{extra,i} = (-1)^i \kappa_Q T_i \hat{\mathbf{z}}_b$$

where κ_Q is the torque coefficient and the sign depends on rotor rotation direction. The control vector becomes:

$$\mathbf{u} = [T_1] \quad (3.26)$$

3.10.3 Step 3: Addition of Remaining Rotors

Additional rotors are added at positions $\mathbf{r}_2$, $\mathbf{r}_3$, and $\mathbf{r}_4$. Each rotor independently contributes force and moment:

$$\mathbf{F}_i = k_T T_i \mathbf{z}_b \quad (3.27)$$

$$\mathbf{M}_i = \mathbf{r}_i \times \mathbf{F}_i + \mathbf{M}_{reaction} \quad (3.28)$$

The control vector expands to:

$$\mathbf{u} = [T_1 \ T_2 \ T_3 \ T_4]^T \quad (3.29)$$

3.10.4 Step 4: Mass and Inertia Definition

The mass and inertia properties are defined at the CG:

$$\mathbf{I} = \begin{bmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{bmatrix} \quad (3.30)$$

These parameters are used in the 6-DOF equations of motion.

3.10.5 Step 5: Force and Moment Aggregation

The total force and moment acting on the system are obtained by summing contributions from all rotors:

$$\mathbf{F}_{net} = \sum_{i=1}^4 \mathbf{F}_i \quad (3.31)$$

$$\mathbf{M}_{net} = \sum_{i=1}^4 \mathbf{M}_i \quad (3.32)$$

This formulation is identical to the general aggregation equations introduced in Chapter 1 and applies without modification.

3.10.6 Failure Scenario

One of the key advantages of the aggregation approach is its ability to naturally capture asymmetric configurations. For example, if one rotor fails:

$$T_3 = 0 \quad (3.33)$$

then:

$$\mathbf{F}_{net} = \mathbf{F}_1 + \mathbf{F}_2 + \mathbf{F}_4 \quad (3.34)$$

$$\mathbf{M}_{net} = \mathbf{M}_1 + \mathbf{M}_2 + \mathbf{M}_4 \quad (3.35)$$

This results in a non-zero net moment, leading to rotational motion without requiring any special-case logic.

This example highlights a fundamental advantage of the proposed framework: no multirotor-specific equations are required at the aircraft level. Each propulsion element contributes independently, and the global behavior emerges from force and moment aggregation. This demonstrates that the same architecture can seamlessly model both fixed-wing aircraft and distributed propulsion systems.

The multirotor example is therefore not just an additional test case; it demonstrates the generality of the architecture. A vehicle with no wings and no conventional control surfaces can still be represented using the same force and moment interface as a fixed-wing aircraft.

3.11 Force and Moment Aggregation

The central idea of the framework is aggregation of all forces and moments, as introduced in Chapter 1. The total force is computed as:

$$\mathbf{F}_{total} = \mathbf{F}_{aero} + \sum_i \mathbf{F}_{prop,i} + \mathbf{F}_{gravity} \quad (3.36)$$

The total moment is computed as:

$$\mathbf{M}_{total} = \mathbf{M}_{aero} + \sum_i (\mathbf{r}_i \times \mathbf{F}_{prop,i} + \mathbf{M}_{extra,i}) + \mathbf{M}_{gravity} \quad (3.37)$$

All contributions are expressed in the body frame prior to aggregation. If forces are evaluated about a reference point different from the center of gravity, gravity may also contribute a moment:

$$\mathbf{M}_{gravity} = \mathbf{r}_{CG/ref} \times \mathbf{F}_{gravity}. \quad (3.38)$$

Equations (3.36)–(3.38) form the core of the aircraft-agnostic architecture. The aircraft may be a fixed-wing aircraft, a transport aircraft, a fighter, or a multirotor, but the final output remains the same: total body-axis force and total body-axis moment.

This formulation eliminates the need to rewrite aircraft-level equations when the configuration changes. The framework modifies only the contributing components, while the aggregation structure remains unchanged. This allows the same simulation engine to represent configurations that would traditionally require separate derivations.

The key contribution of this work is the formulation of an aircraft-agnostic force and moment aggregation framework, in which all subsystem contributions are independently computed and consistently expressed in a common reference frame prior to integration with a unified 6-DOF dynamics model.

$$m(\dot{u} + qw - rv) = F_x \quad (3.39)$$

$$m(\dot{v} + ru - pw) = F_y \quad (3.40)$$

$$m(\dot{w} + pv - qu) = F_z \quad (3.41)$$

$$I_{xx}\dot{p} - (I_{yy} - I_{zz})qr = M_x \quad (3.42)$$

$$I_{yy}\dot{q} - (I_{zz} - I_{xx})pr = M_y \quad (3.43)$$

$$I_{zz}\dot{r} - (I_{xx} - I_{yy})pq = M_z \quad (3.44)$$

This eliminates the need for configuration-specific equations and enables seamless modeling of fundamentally different aircraft types within a single simulation architecture.

3.12 Architecture Summary

The framework separates aircraft definition from simulation logic and ensures that all configurations produce consistent outputs: total body-axis force and moment. These outputs are used by all analysis modules, including trim, performance, stability, mission planning,

and simulation.

The key architectural benefit is that new aircraft configurations can be created by changing the components registered in the aircraft object, rather than rewriting the aircraft-level equations of motion. This unified structure enables the same framework to support fundamentally different aircraft configurations without modifying the core simulation pipeline.

In summary, the framework is built on three core architectural ideas: a centralized aircraft object, modular subsystem definitions, and force/moment aggregation in a common reference frame. Together, these ideas enable a reusable simulation environment that can support conventional aircraft, high-performance jets, transport aircraft, and rotor-based systems within a single modeling structure.

Chapter 4

Analysis and Simulation

The primary advantage of the proposed framework is that all analysis and simulation capabilities operate on a single, consistent aircraft definition. Unlike traditional workflows where trim, performance, stability, and simulation are treated as independent processes, this framework unifies them through a common force and moment evaluation pipeline.

All modules rely on the same underlying aggregation of aerodynamic, propulsive, and gravitational forces and moments, as defined in Eqs. (3.36)–(3.37). This ensures that results obtained from trim, performance analysis, stability analysis, and time-domain simulation remain physically consistent. This consistency is important because trim, linearization, and simulation should all represent the same nonlinear aircraft model rather than separate approximations [1], [2].

This chapter describes the implementation of trim analysis, performance evaluation, stability analysis, mission-level simulation, takeoff and landing analysis, and real-time simulation using a generalized Simulink S-function interface.

4.1 Trim Analysis

Trim analysis determines steady-state flight conditions in which the aircraft experiences zero net force and moment. Physically, this corresponds to equilibrium flight where translational and rotational accelerations are zero. Trim is a fundamental step in aircraft analysis because performance calculations, stability linearization, and time-domain perturbation studies are usually performed about a known equilibrium condition [1, 4].

The equilibrium condition is defined as:

$$\sum \mathbf{F}(\mathbf{x}, \mathbf{u}) = \mathbf{0}, \quad \sum \mathbf{M}(\mathbf{x}, \mathbf{u}) = \mathbf{0} \quad (4.1)$$

These equations represent six nonlinear constraints in the unknown state and control variables. Because aerodynamic forces depend nonlinearly on angle of attack, velocity, atmospheric density, and control deflections, a closed-form solution is generally not possible. For symmetric cruise trim, the formulation is:

- Free variables: $\alpha, \theta, \delta_e, \delta_t$
- Constraints: $F_z = 0, M_x = 0, M_y = 0, V = \text{constant}$
- Specified: altitude h , flight path angle $\gamma = 0$

Instead, trim is formulated as a nonlinear optimization problem:

$$\min_{\mathbf{x}, \mathbf{u}} J(\mathbf{x}, \mathbf{u}) \quad (4.2)$$

where the objective function J is typically defined as the norm of residual forces and moments, or alternatively as a penalty on control usage or power consumption. A representative residual-based objective may be written as:

$$J(\mathbf{x}, \mathbf{u}) = \left\| \begin{bmatrix} \sum \mathbf{F}(\mathbf{x}, \mathbf{u}) \\ \sum \mathbf{M}(\mathbf{x}, \mathbf{u}) \end{bmatrix} \right\|_2 \quad (4.3)$$

Typical constraints include:

- Fixed altitude
- Prescribed velocity or Mach number
- Flight path angle γ

The solver uses gradient-based methods such as Sequential Quadratic Programming (SQP) through `fmincon`. In cases where derivatives are unreliable, derivative-free solvers are used. The use of numerical optimization is appropriate because the framework supports multiple aircraft types and subsystem models, making analytical trim equations configuration-dependent.

At each iteration:

1. State and control are updated
2. Aerodynamic and propulsion models are evaluated
3. Total force and moment are computed using Eqs. (3.36)–(3.37)

4. Residuals are minimized until the force and moment balance satisfies the convergence tolerance

Because the same force and moment functions are used in both trim and simulation, the resulting trim condition is dynamically consistent. This is one of the major advantages of using a unified aircraft object rather than separate trim and simulation models.

4.2 Performance Analysis

Performance analysis evaluates aircraft capability across a range of operating conditions, typically defined by altitude and velocity grids. The objective is to determine how available thrust, aerodynamic drag, aircraft weight, and atmospheric conditions affect climb capability, cruise behavior, and energy performance [4].

At each operating point (h, V) , aerodynamic and propulsion models are evaluated to compute:

- Lift L
- Drag D
- Thrust available T
- Rate of climb (ROC)
- Specific excess power P_s

The rate of climb is defined as:

$$ROC = \frac{(T - D)V}{W} \quad (4.4)$$

This expression assumes steady climb conditions with a small flight path angle. The numerator $(T - D)V$ represents excess power, and division by weight converts this excess power into a climb rate.

The specific excess power is:

$$P_s = \frac{(T - D)V}{W} \quad (4.5)$$

Under steady climb conditions with approximately constant velocity, the numerical value of specific excess power becomes equal to the rate of climb. More generally, specific excess power represents the excess energy available for climbing or acceleration.

Two modes are supported:

- Fixed α analysis, useful for parametric sweeps and aerodynamic sensitivity studies
- Lift-balanced mode, where α is solved such that $L = W$, representing steady level-flight conditions

Because propulsion is modeled per component, thrust variation with altitude, Mach number, and throttle is naturally captured. This is particularly important for modern aircraft with distributed propulsion, where total thrust and total moment depend on the contribution of each propulsion element rather than a single aggregate thrust value.

Outputs from the performance module can include rate-of-climb envelopes, thrust-required and thrust-available curves, drag trends, fuel-flow estimates, and operating points that maximize climb performance or endurance. These outputs are useful both for validation of known aircraft and for early-stage design comparison of new configurations.

4.3 Stability Analysis

Stability analysis is performed by linearizing the nonlinear equations of motion about a trim condition. This provides a local approximation of the aircraft dynamics and allows classical dynamic modes to be identified using eigenvalue analysis [1, 2, 3].

The system is expressed in state-space form:

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u} \quad (4.6)$$

where A is the state matrix and B is the input matrix. In the framework, these matrices are obtained numerically using finite differences:

$$A = \frac{\partial \dot{\mathbf{x}}}{\partial \mathbf{x}}, \quad B = \frac{\partial \dot{\mathbf{x}}}{\partial \mathbf{u}} \quad (4.7)$$

For a state perturbation $\Delta \mathbf{x}_i$, the corresponding column of A may be approximated using a central difference: The perturbation size is chosen as:

$$\Delta x_i = \max(|x_i| \sqrt{\epsilon_{mach}}, 10^{-6})$$

This scaling ensures numerical stability by balancing truncation error and floating-point precision.

$$A_i \approx \frac{\dot{\mathbf{x}}(\mathbf{x}_0 + \Delta \mathbf{x}_i, \mathbf{u}_0) - \dot{\mathbf{x}}(\mathbf{x}_0 - \Delta \mathbf{x}_i, \mathbf{u}_0)}{2\Delta x_i} \quad (4.8)$$

A similar procedure is used to compute the columns of B by perturbing each control input. This numerical approach is useful because the framework supports arbitrary aerodynamic

and propulsion models, which may not have analytical derivatives. The eigenvalues of the state matrix A ,

$$\lambda = \sigma \pm j\omega \quad (4.9)$$

determine the dynamic behavior and stability of the system. For oscillatory modes, the natural frequency and damping ratio can be computed as:

$$\omega_n = \sqrt{\sigma^2 + \omega^2}, \quad \zeta = -\frac{\sigma}{\omega_n} \quad (4.10)$$

The following classical modes are identified:

- Short-period mode, representing fast pitch dynamics
- Phugoid mode, representing long-period exchange between kinetic and potential energy
- Dutch roll, representing coupled lateral-directional oscillation
- Roll subsidence, representing roll-rate damping
- Spiral mode, representing slow lateral-directional divergence or convergence

Static stability derivatives are also evaluated:

$$C_{m_\alpha} = \frac{\partial C_m}{\partial \alpha}, \quad C_{n_\beta} = \frac{\partial C_n}{\partial \beta} \quad (4.11)$$

These quantities determine longitudinal and directional static stability respectively. A negative C_{m_α} generally indicates longitudinal static stability, while a positive C_{n_β} generally indicates directional static stability under the sign conventions used in this framework.

4.4 Mission Planning and Trim Scheduling

Mission-level simulation is implemented using a segmented approach. Each mission consists of multiple flight phases such as climb, cruise, descent, approach, or hold. This structure allows complex trajectories to be represented as a sequence of simpler operating conditions.

Each segment defines:

- Target altitude
- Mach number or velocity
- Flight path angle

- Duration

Trim is computed at discrete points along the mission trajectory. To ensure smooth transitions between segments, previously computed trim states are used as initial guesses for subsequent segments. This reduces computational cost and improves convergence robustness because neighboring flight conditions usually have similar equilibrium states.

The result is a continuous profile of:

- Aircraft states
- Control inputs
- Performance metrics

The mission planner therefore acts as a bridge between static trim analysis and time-domain simulation. Instead of trimming only one isolated operating point, the framework generates a schedule of trimmed states and controls across a complete flight profile. This is useful for simulating realistic missions and for initializing Simulink time histories.

4.5 Takeoff Analysis

Takeoff performance is modeled using longitudinal ground-roll dynamics. The governing equation is:

$$m \frac{dV}{dt} = T - D - \mu(W - L) \quad (4.12)$$

This formulation represents the balance of forces along the runway direction and includes thrust acceleration, aerodynamic drag, and rolling resistance. The lift term reduces the normal force on the wheels, thereby reducing rolling resistance as speed increases.

Assumptions. The takeoff model is based on the following assumptions:

- Motion is constrained to one-dimensional longitudinal dynamics along the runway.
- The runway is level, and slope effects are neglected.
- Aerodynamic forces are evaluated using quasi-steady assumptions.
- The rolling friction coefficient μ is assumed constant.
- Thrust is aligned with the runway direction and varies slowly with speed.

- Wind effects and runway surface variations are not considered.

The ground-roll distance is obtained by numerical integration:

$$S_{TO,ground} = \int_0^{t_R} V(t) dt \quad (4.13)$$

where t_R is the time required to reach rotation speed.

To report a more complete takeoff distance, the airborne segment from rotation to a specified screen height is also included:

$$S_{TO,total} = S_{TO,ground} + S_{airborne} \quad (4.14)$$

For transport-category style reporting, the screen height is commonly taken as 35 ft above the runway. In this work, the screen height is used as a reference distance metric rather than a certification claim.

Important characteristic speeds include:

- V_s (stall speed)
- V_R (rotation speed)
- V_2 (safe climb speed)

Balanced Field Length (BFL) is evaluated under one-engine-inoperative (OEI) conditions by comparing the accelerate-go and accelerate-stop distances. The BFL corresponds to the runway length at which the continue-takeoff distance and rejected-takeoff distance are equal, or the larger of the two when evaluated at a selected decision speed V_1 . This allows the takeoff module to assess engine-out performance margins in addition to nominal performance.

4.6 Landing Analysis

Landing performance is modeled as a combination of approach, flare, and ground-roll phases. The total landing distance is expressed as:

$$S_{landing} = S_{approach} + S_{flare} + S_{ground} \quad (4.15)$$

The approach phase is measured from a specified screen height to the flare initiation point. For transport-category style reporting, a 50 ft screen height is commonly used. In this work, this height is used as a consistent reference for comparing landing distance predictions.

The approach phase is governed by glide slope and descent conditions, while the flare phase reduces the descent rate prior to touchdown. The ground-roll phase includes aerodynamic drag, braking forces, and rolling resistance.

Assumptions. The landing model is based on the following assumptions:

- The approach is flown at a constant glide slope.
- Flare is modeled as a smooth transition without detailed pitch dynamics.
- Thrust is reduced to idle during ground roll.
- Braking is represented using an effective friction coefficient.
- Aerodynamic forces are evaluated using quasi-steady assumptions.
- Runway conditions are assumed uniform.

The ground-roll portion is evaluated using a force balance similar to the takeoff case, with thrust reduced and braking forces included. Due to the nonlinear dependence on velocity, lift, drag, and braking, numerical integration is used to compute the ground-roll distance.

This formulation allows the landing analysis to account for variations in aircraft weight, approach speed, braking effectiveness, and reference screen height within a consistent simulation framework.

4.7 Time-Domain Simulation

The aircraft is simulated using 6-DOF rigid-body equations of motion. The translational dynamics are represented as:

$$m\dot{\mathbf{V}} = \mathbf{F}_{aero} + \mathbf{F}_{prop} + \mathbf{F}_{gravity} + \mathbf{F}_{other} \quad (4.16)$$

and the rotational dynamics are:

$$\mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) = \mathbf{M}_{total} \quad (4.17)$$

These equations capture translational and rotational motion of the aircraft. The nonlinear coupling term $\boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega})$ represents gyroscopic coupling due to angular velocity and inertia. This formulation follows standard rigid-body aircraft dynamics [1, 2, 4].

In implementation, the total forces and moments are supplied by the aggregation framework described in Chapter 3. Therefore, the 6-DOF equations do not require aircraft-specific

modifications. The same dynamics block can be used for fixed-wing aircraft, fighter aircraft, transport aircraft, and multirotor vehicles as long as the aircraft object provides compatible total force and moment vectors.

Simulation is performed in Simulink using fixed-step solvers, enabling real-time execution and hardware-in-the-loop (HIL) capability. Fixed-step integration is important for real-time applications because the simulation must advance at deterministic time intervals.

4.8 Generalized Simulink S-Function Interface

The S-function provides a bridge between MATLAB object-oriented models and Simulink numerical integration. It allows the modular aircraft object to be evaluated inside a Simulink simulation without rewriting the object-oriented model as blocks.

It performs:

- Control input mapping
- Rate limiting
- Force/moment computation
- State propagation

At each time step, the S-function receives the current aircraft state and control vector, updates the aircraft object, evaluates all subsystem contributions, and outputs total forces and moments to the 6-DOF block. This creates a direct connection between the MATLAB aircraft framework and Simulink real-time simulation.

4.8.1 Rate Limiting

Rate limiting is included to represent actuator dynamics and prevent unrealistic instantaneous changes in control input. The discrete update may be written as:

$$u_{k+1} = u_k + \text{clip}(u_{desired} - u_k, \pm \Delta u_{max}) \quad (4.18)$$

where u_k is the current control value, $u_{desired}$ is the commanded value, and Δu_{max} is the maximum allowed input change per time step. This is important for realistic control surface motion, throttle response, or rotor command scheduling.

4.9 Unified Analysis Pipeline

All modules rely on the same force and moment evaluation:

- Trim solves Eq. (4.1)
- Performance uses Eqs. (4.4)–(4.5)
- Stability uses Eqs. (4.6)–(4.9)
- Simulation integrates Eqs. (4.16)–(4.17)

This unified pipeline ensures that all analyses are consistent and derived from a single physical model, which is the core contribution of this work. The same aircraft definition can be used to compute equilibrium conditions, generate performance maps, evaluate stability modes, build mission profiles, and run time-domain simulations. This reduces model duplication and makes the framework extensible to new aircraft configurations.

Chapter 5

Results and Validation

This chapter presents a comprehensive validation of the proposed aircraft-agnostic modular framework across multiple aircraft configurations spanning distinct aerodynamic and propulsion regimes. The objective of this validation is not only to demonstrate numerical convergence, but also to establish physical consistency, scalability, and generality of the framework.

This framework was validated using four representative configurations:

- Cessna 172 (low-speed propeller-driven aircraft)
- F-16 (high-performance fighter aircraft)
- Boeing 737 (transport aircraft using DATCOM-based [30] aerodynamics)
- Quadrotor (distributed propulsion multirotor system)

These cases were selected to evaluate the framework across fundamentally different flight physics while maintaining a single modeling architecture. All simulations utilize the same force and moment aggregation formulation defined in Eqs. (3.36)–(3.37), ensuring consistency across results..

5.1 Validation Methodology

The validation approach is based on three criteria: numerical convergence, physical consistency, and cross-configuration generality. Numerical convergence is evaluated using force and moment residuals from trim solutions. Physical consistency is assessed by examining whether the resulting trim states, control inputs, stability modes, and performance values follow expected aircraft behavior. Cross-configuration generality is evaluated by applying the same framework to fixed-wing, high-performance jet, transport, and multirotor configurations.

For cases where reference data are available, the percent error is computed as:

$$\text{Error} = \left| \frac{X_{sim} - X_{ref}}{X_{ref}} \right| \times 100 \quad (5.1)$$

where X_{sim} is the simulated value and X_{ref} is the reference value. This metric provides a normalized measure of deviation between simulated and reference values, enabling comparison across different performance quantities. The validation presented in this chapter is not intended to certify the models, but rather to demonstrate that the proposed architecture produces physically meaningful results across multiple aircraft types. Classical flight dynamics references are used to interpret stability and performance trends[1, 2]

5.2 Cessna 172 Configuration and Validation

The Cessna 172 serves as a baseline validation case for steady-state performance and trim consistency in a conventional fixed-wing aircraft.

5.2.1 Configuration

The configuration parameters are:

- Empty mass: 767 kg
- Fuel mass: 90 kg
- Total mass: 857 kg
- Wing area: 16.17 m²
- Wing span: 10.98 m
- Mean aerodynamic chord: 1.50 m
- Engine: Lycoming O-360 (180 HP)

Reference performance data was obtained from Air Plains conversion data for the 180 HP configuration.

5.2.2 Cruise Performance Comparison

Fuel flow and brake horsepower predictions show agreement within approximately 5%, which is acceptable for a conceptual-level model.

Table 5.1: Cessna 172 cruise performance validation
[29]

Condition	Ref KTAS	Ref GPH	Sim GPH	Fuel Error (%)	Ref BHP	Sim BHP	BHP Error (%)
2000 ft cruise	113.0	9.2	8.8	-4.69	68	65.2	-2.80
6000 ft cruise	112.0	8.8	8.4	-4.45	65	61.3	-3.74
10000 ft cruise	110.0	8.1	8.0	-1.20	59	56.9	-2.14
6000 ft economy	103.0	7.9	7.6	-3.43	50	53.1	3.11

Detailed Trim Example (6000 ft)

- Angle of attack: 2.75°
- Pitch angle: 2.75°
- Elevator: -0.58°
- Throttle: 0.528

Performance:

- Fuel error: -4.4%
- BHP error: -3.7%
- Thrust power: 57.3 HP
- Shaft power: 110.3 HP

These values indicate physically consistent propulsion–aerodynamic coupling and realistic propeller efficiency behavior. In addition to matching the prescribed airspeed, validation is performed by comparing trim control variables including elevator deflection, throttle setting, and angle of attack against reference values.

5.2.3 Trim Validation

Trim converged with negligible residuals:

- Force residual: 7.41×10^{-4} N
- Moment residual: 3.48×10^{-4} N-m

These residual values confirm that the trim solution satisfies force and moment equilibrium within numerical tolerance, indicating successful convergence.

5.2.4 Center of Gravity Migration

CG movement due to fuel burn:

- Fuel 180 kg: $[-0.039, 0.000, 0.062]$ m
- Fuel 50 kg: $[-0.015, -0.092, 0.024]$ m
- Fuel 0 kg: $[0.000, 0.000, 0.000]$ m

These results demonstrate that fuel burn leads to noticeable shifts in center of gravity, which can influence trim conditions, stability characteristics, and control requirements.

5.3 F-16 Performance Validation and Envelope Analysis

Trim across representative flight regimes is summarized in Table 5.2. These conditions span subsonic cruise, climb, and supersonic dash, demonstrating the ability of the framework to converge trim solutions across a wide flight envelope.

Table 5.2: F-16 trim results across representative flight conditions

Condition	Altitude (m)	Mach	α ($^\circ$)
Cruise	9144	0.60	6.92
Climb	5000	0.40	8.96
Dash	10000	1.60	-0.27

The variation in angle of attack reflects expected aerodynamic behavior across regimes. Higher α is required during climb due to increased lift demand, while near-zero or slightly negative α is observed in high-speed dash conditions due to increased dynamic pressure and compressibility effects.

5.3.1 Stability Analysis

The system is linearized about the trim condition to obtain state-space matrices:

$$A \in \mathbb{R}^{12 \times 12}, \quad B \in \mathbb{R}^{12 \times 4}$$

The dominant dynamic modes are:

- Phugoid: 33.13 s (long-period, lightly damped mode)
- Short-period: 2.26 s (fast, well-damped longitudinal mode)

- Roll subsidence: 0.756 s (stable lateral damping mode)

Table 5.3: F-16 stability mode comparison

Mode	Simulation	Reference (NASA TP-1538)
Short-period	2.26 s	2.1 s
Phugoid	33.13 s	35.2 s
Roll subsidence	0.756 s	0.8 s

These values are consistent with expected fighter aircraft behavior and align with characteristics observed in relaxed-stability aircraft configurations [6]. The short-period mode governs pitch response, while the phugoid represents energy exchange between kinetic and potential energy [1].

5.4 Mission Planning Validation

- Duration: 2172.8 s
- Points: 8697

The trim solver successfully generates a continuous mission trajectory, demonstrating the ability of the framework to maintain consistent equilibrium solutions across changing flight conditions.

5.4.1 Rate of Climb as a Function of Velocity

The rate of climb is computed using Eq. (4.4) Figure 5.1 shows the variation of rate of climb with velocity at sea level.

The rate of climb initially increases with velocity as excess thrust increases. At low speeds, the aircraft operates in a drag-dominated regime, limiting climb performance. As velocity increases, aerodynamic efficiency improves, leading to a rapid increase in ROC.

A peak rate of climb of approximately 200–210 m/s is observed near 260–280 m/s. This corresponds to approximately 39,000–41,000 ft/min, which lies within reported F-16 climb rate ranges (typically 40,000–55,000 ft/min) [8, 7, 5]. The slightly lower predicted value relative to upper-bound references can be attributed to simplified propulsion modeling and the absence of detailed engine performance maps.

Beyond this optimal speed, ROC decreases due to increasing drag and reduction in excess thrust at higher velocities.

At very high velocities, thrust required approaches thrust available, resulting in near-zero climb capability.

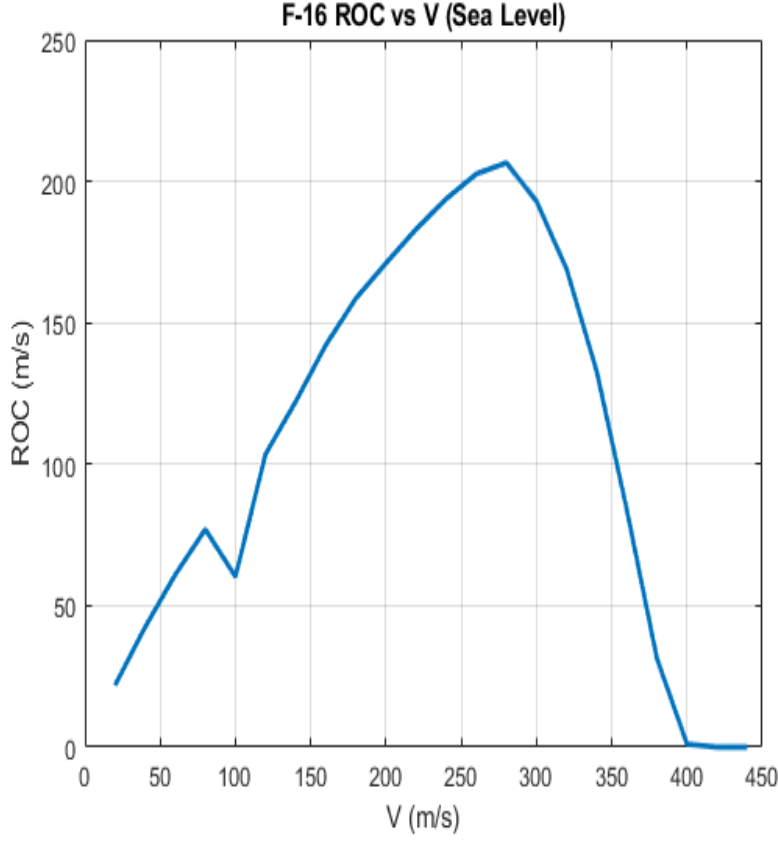


Figure 5.1: Rate of climb variation with velocity for the F-16 at sea level.

5.4.2 Rate of Climb Envelope with Altitude

Figure 5.2 shows the variation of maximum rate of climb with altitude.

The maximum rate of climb decreases monotonically with altitude due to reductions in air density and engine thrust.

At lower altitudes, higher air density improves both propulsion efficiency and aerodynamic lift generation. As altitude increases, both thrust available and aerodynamic effectiveness decrease, leading to reduced climb performance.

The service ceiling corresponds to the altitude at which maximum ROC approaches zero. The service ceiling is observed near 18–19 km, defined as the altitude at which the maximum achievable rate of climb approaches zero [7, 5].

5.4.3 Thrust Available vs Thrust Required

The difference between thrust available and thrust required represents excess thrust, which directly governs climb performance.

At low velocities, thrust required is low, but aerodynamic inefficiencies limit performance.

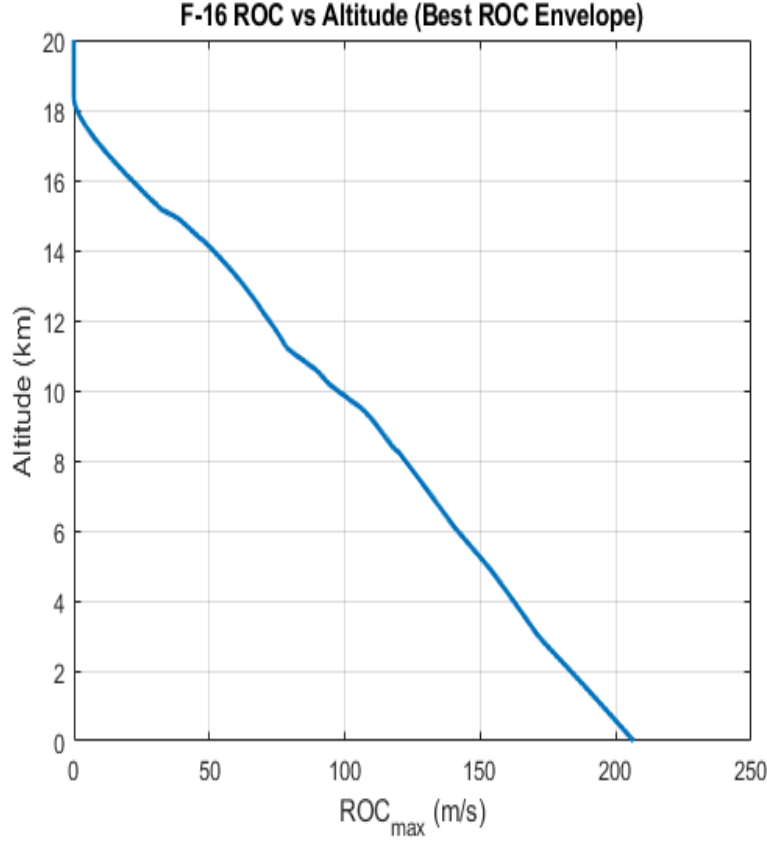


Figure 5.2: Maximum rate of climb envelope as a function of altitude.

As velocity increases, thrust required initially increases gradually, while thrust available remains relatively high, resulting in a region of maximum excess thrust.

At higher velocities, drag increases rapidly, causing thrust required to approach and eventually exceed thrust available. This reduces excess thrust and consequently reduces rate of climb.

The intersection point of the two curves represents the maximum achievable velocity at the given altitude, where excess thrust becomes zero.

This relationship highlights that climb performance is fundamentally governed by excess thrust, linking propulsion capability and aerodynamic efficiency [28].

5.4.4 Key Observation

The results demonstrate that the proposed force and moment aggregation framework is capable of capturing realistic high-performance aircraft behavior across multiple flight regimes. While the aerodynamic and propulsion models are simplified, the framework produces physically consistent trends in trim, stability, and performance.

The simulated ceiling and climb trends align with publicly available performance data,

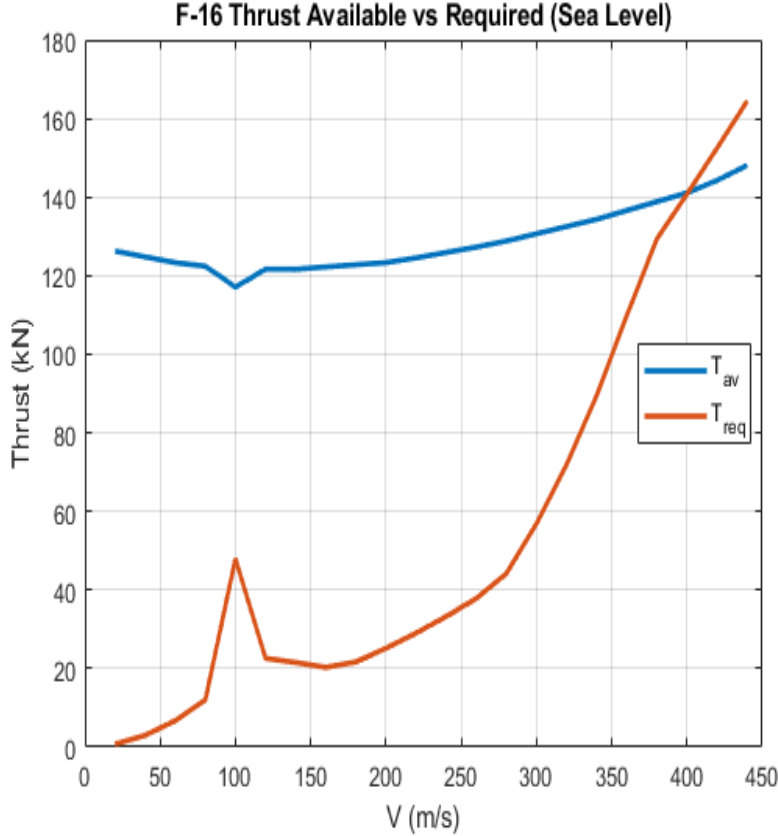


Figure 5.3: Thrust available and thrust required as a function of velocity.

though absolute rate-of-climb values are slightly over predicted due to simplified propulsion modeling.. Publicly available F-16 performance data varies depending on configuration, payload, and engine variant [7, 5]; therefore, comparisons are made at the level of performance envelopes rather than exact values.

5.5 Boeing 737 DATCOM Validation

A Boeing 737 configuration was modeled using aerodynamic data derived from DATCOM tables [30]. A total of 180 aerodynamic tables were extracted, spanning Mach numbers from 0.1 to 0.6 and angles of attack from -16° to 22° , with 16 discretization points in angle of attack for each Mach condition. The configuration corresponds to a wing-body-vertical tail-horizontal tail setup.

5.5.1 Trim and Aerodynamic Performance

Trim analysis was performed at Mach 0.6 and an altitude of 7000 m. The solver successfully converged to a physically consistent solution:

- Angle of attack: 6.22°
- Elevator deflection: -4.61°
- Throttle setting: 0.404

The corresponding aerodynamic coefficients are:

- $C_L = 0.4957$
- $C_D = 0.0271$
- Lift-to-drag ratio: $L/D = 18.3$

These values are consistent with expected transport aircraft performance in subsonic cruise, indicating realistic aerodynamic behavior from the DATCOM-based model.

5.5.2 Takeoff Performance

Takeoff performance was evaluated using a balanced field length (BFL) analysis under a symmetric multi-engine configuration:

- Stall speed: $V_{stall} = 75.4$ m/s
- Rotation speed: $V_R = 83.0$ m/s
- Safety climb speed: $V_2 = 90.5$ m/s
- Decision speed: $V_1 = 76.5$ m/s

The computed takeoff distance is:

- Balanced field length: 2217 m

The close agreement between go and stop distances confirms consistency in the takeoff model.

5.5.3 One-Engine-Inoperative (OEI) Climb Performance

Climb gradients were evaluated under OEI conditions:

- Second segment climb gradient (gross): 9.55% (requirement: 2.4%)
- Final climb gradient (gross): 9.88% (requirement: 1.2%)

All regulatory constraints are satisfied, indicating adequate thrust modeling and realistic climb performance margins.

5.5.4 Landing Performance

Landing distance was computed using approach, flare, and ground roll phases:

- Total landing distance: 1598.5 m
- Stall speed (landing): 61.9 m/s
- Approach speed: 80.4 m/s
- Touchdown speed: 70.8 m/s

The results fall within expected ranges for transport-category aircraft.

5.5.5 Stability Analysis

Static stability derivatives confirm expected aircraft behavior:

- $C_{m_\alpha} = -2.30 / rad$ (longitudinally stable)
- $C_{n_\beta} = 0.09 / rad$ (directionally stable)

Dynamic stability modes were identified through eigenvalue analysis:

- Short-period: $\omega_n = 1.71$, $\zeta = 0.18$ (stable)
- Phugoid: $\omega_n = 0.082$, $\zeta = 0.024$ (lightly damped, stable)
- Dutch roll: $\omega_n = 0.75$, $\zeta = 0.048$ (weakly damped)
- Roll subsidence: $\tau = 2.18$ s (stable)
- Spiral mode: $\tau = 235.7$ s (unstable)

This behavior is consistent with FAA Part 25 handling qualities, which allow slow spiral divergence provided the time-to-double exceeds certification thresholds. The presence of a weakly unstable spiral mode is consistent with classical transport aircraft behavior and has been widely reported in literature [10]. This mode is typically stabilized through automatic control systems.

5.5.6 Key Observation

The results demonstrate that the DATCOM-based aerodynamic model [30], when integrated into the force and moment aggregation framework, produces physically consistent trim, performance, and stability characteristics across multiple flight regimes. This validates the ability of the framework to handle transport-class aircraft without requiring configuration-specific modifications.

5.6 Quadrotor Validation

- Mass: 1.5 kg
- Rotor diameter: 0.254 m

Successful:

- Takeoff
- Hover
- Landing

5.7 Computational Performance Analysis

Table 5.4: MATLAB profiler summary for Cessna 172 trim computation

Component	Representative Function	Calls	Time (s)
Trim Solver	<code>solve_cruise_trim</code>	1	0.585
Optimization	<code>fmincon</code>	1	0.496
Residual Evaluation	<code>trim_residual_symmetric</code>	91	0.336
External Forces	<code>calculate_external_forces_moments</code>	94	0.267
Aerodynamics	<code>calculate_forces_moments</code>	94	0.152
Atmosphere Model	<code>atmosisa</code>	283	0.131
Propulsion	<code>get_force_moment</code>	94	0.057
Control Mapping	<code>set_full_controls</code>	98	0.120

To evaluate the computational efficiency of the proposed framework, MATLAB profiler was used to analyze execution time distribution during a representative trim computation for the Cessna 172 configuration.

The total execution time for the simulation was approximately 1.129 s. The results indicate that the dominant computational cost is associated with nonlinear trim optimization. The `solve_cruise_trim` routine required approximately 0.585 s, with the underlying `fmincon` optimizer accounting for 0.496 s of the total runtime.

During optimization, the trim residual function was evaluated 91 times, resulting in repeated calls to the force and moment evaluation pipeline. This residual evaluation alone accounted for approximately 0.336 s.

The force and moment evaluation routines required approximately 0.277 s over 94 calls. Within this pipeline, aerodynamic force and moment computation contributed 0.152 s, while atmospheric property evaluation using `atmosisa` accounted for 0.131 s over 283 calls. Propulsion force computation contributed a smaller portion of the total runtime at 0.057 s.

Control vector mapping and synchronization required approximately 0.120 s, indicating moderate overhead associated with control handling.

Importantly, the aggregation of forces and moments itself does not appear as a dominant contributor in the profiler results. This demonstrates that the modular architecture introduces negligible computational overhead.

Overall, the results confirm that the computational cost is primarily driven by nonlinear optimization and repeated physical model evaluation, rather than by the structure of the framework itself. This behavior is desirable, as increased model fidelity directly affects computational cost while maintaining architectural efficiency.

Chapter 6

Real-Time Simulation and Visualization

The proposed framework is extended beyond offline analysis by integrating the MATLAB/Simulink aircraft model with a real-time visualization environment. This enables the same aircraft model used for trim, performance, and stability analysis to be executed in a simulator-style environment with continuous visual feedback.

This capability demonstrates that the framework supports not only analysis but also interactive simulation and visualization workflows, effectively bridging the gap between conceptual design tools and real-time flight simulation systems [1, 11]. Unlike traditional workflows where analysis and simulation models are decoupled, the same unified aircraft object is used directly within a real-time simulation loop.

6.1 Simulation Hardware Setup

Figure 6.1 shows the physical simulation setup used for testing. The system includes a pilot input station, display interfaces, and an external visualization screen. MATLAB/Simulink is used as the simulation backend, while aircraft state data is streamed to an external visualization client in real time.

This setup enables pilot-in-the-loop interaction while maintaining a physics-based simulation backend. Such configurations are commonly used in flight simulation environments for control validation and human-in-the-loop testing, where real-time feedback is critical for system evaluation.



Figure 6.1: Physical simulation setup used for real-time flight visualization and pilot-in-the-loop testing.

6.2 Simulink S-Function and 6-DOF Integration

The aircraft model is integrated with Simulink using a generalized Level-2 MATLAB S-function. The S-function acts as a bridge between the object-oriented aircraft model and the numerical integration environment.

At each simulation time step, the following sequence is executed:

1. The current state vector $\mathbf{x}$ and control vector $\mathbf{u}$ are passed into the S-function
2. The aircraft object updates all subsystem states (aerodynamics, propulsion, mass)
3. Total forces and moments are computed using the aggregation framework
4. These forces and moments are passed to the 6-DOF rigid-body dynamics block

The governing rigid-body equations solved in Simulink follow the standard 6-DOF formulation:

$$m\dot{\mathbf{V}} = \mathbf{F}_{body} - m\boldsymbol{\omega} \times \mathbf{V} \quad (6.1)$$

$$\mathbf{I}\dot{\boldsymbol{\omega}} = \mathbf{M}_{body} - \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) \quad (6.2)$$

where $\mathbf{V}$ is the body velocity vector and $\boldsymbol{\omega}$ is the angular velocity vector.

Figure 6.2 illustrates this integration.

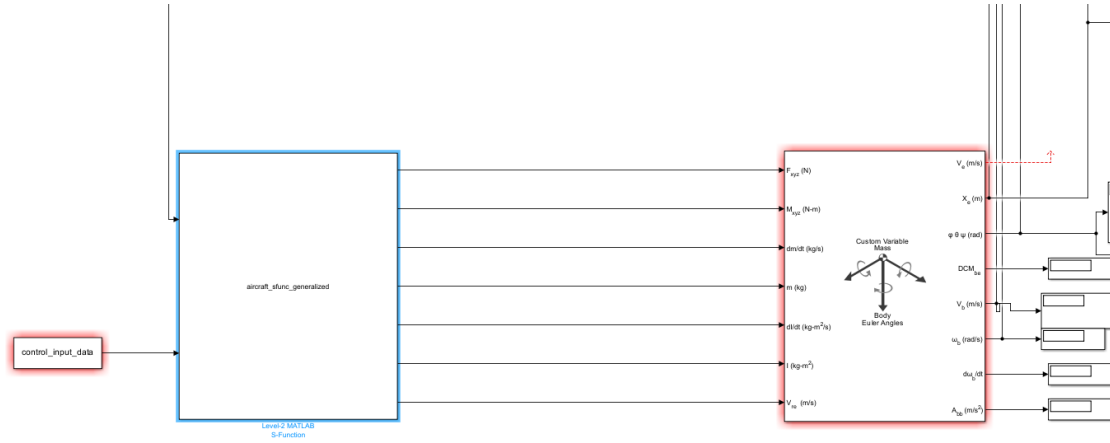


Figure 6.2: Integration of the generalized aircraft S-function with the 6-DOF rigid-body dynamics block.

A fixed-step solver (`ode4`) is used to ensure deterministic execution timing, which is essential for real-time simulation, hardware-in-the-loop (HIL), and control system testing [15].

A key advantage of this design is that the S-function remains **aircraft-agnostic**. Any aircraft configuration—fixed-wing, jet, or multirotor—can be simulated without modifying the Simulink model.

6.3 Real-Time Visualization Pipeline

The outputs of the 6-DOF simulation are converted into a visualization-compatible format and streamed to an external client in real time.

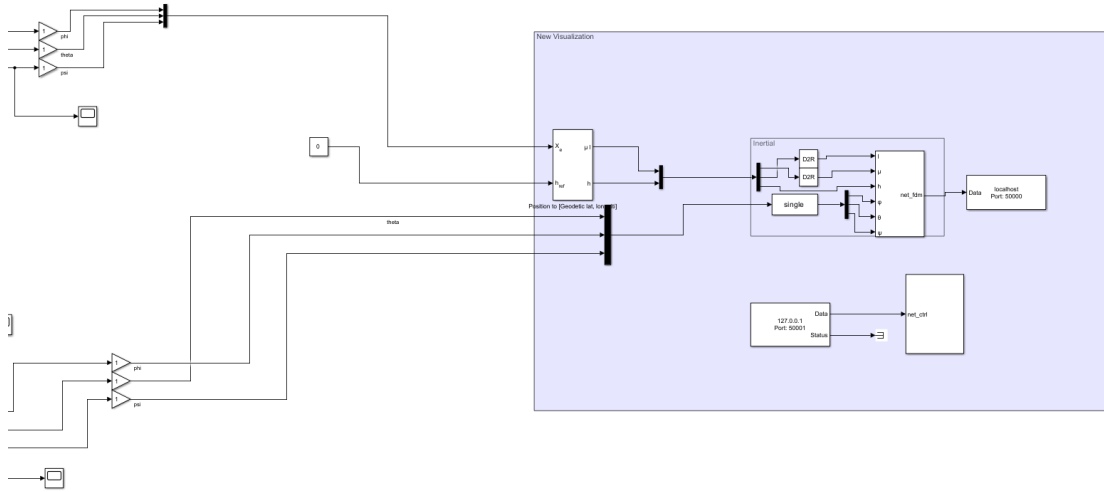


Figure 6.3: Real-time visualization pipeline converting 6-DOF states into external client data.

The pipeline performs the following transformations:

- Conversion from NED coordinates to geodetic coordinates (latitude, longitude, altitude)
- Transformation of Euler angles into visualization-compatible orientation representations
- Packaging of simulation states into structured data formats (JSON)
- Streaming of data via UDP/WebSocket protocols

The coordinate transformation is essential since simulation is performed in the NED frame, while visualization systems typically require Earth-referenced coordinates.

This separation ensures that the simulation core remains independent of visualization constraints.

6.4 Decoupled Simulation and Visualization

A key architectural decision in this framework is the strict separation of aircraft physics from visualization.

The MATLAB/Simulink environment computes aircraft dynamics, while the visualization system handles rendering and user interaction.

This decoupling provides several advantages:

- Independent verification and validation of physics and visualization layers
- Flexibility to integrate multiple visualization platforms (Cesium, Unreal, X3D)
- Scalability to distributed simulation architectures
- Reduced coupling between numerical solvers and rendering pipelines

This design philosophy aligns with modern flight simulation architectures such as JSBSim and FlightGear, where physics engines and visualization engines operate as independent systems [11].

6.5 Platform-Independent Visualization Bridge

To enable flexible visualization, a platform-independent communication bridge was implemented using Python. This bridge connects MATLAB/Simulink to web-based visualization clients.

The data flow architecture is:

$$\text{Simulink} \rightarrow \text{UDP} \rightarrow \text{Python Bridge} \rightarrow \text{WebSocket} \rightarrow \text{Visualization Client} \quad (6.3)$$

The Python bridge performs:

- Receiving aircraft state data via UDP sockets
- Decoding binary packets into structured data fields
- Converting data into JSON format
- Broadcasting updates via WebSocket server
- Hosting an HTTP server for visualization interface delivery

This architecture is implemented using a multithreaded design with asynchronous queues, enabling continuous bidirectional communication between MATLAB and the visualization client.

The visualization client (e.g., CesiumJS) receives:

$$\mathbf{p}_{LLA} = \begin{bmatrix} \text{latitude} & \text{longitude} & \text{altitude} \end{bmatrix} \quad (6.4)$$

$$\boldsymbol{\eta} = \begin{bmatrix} \phi & \theta & \psi \end{bmatrix} \quad (6.5)$$

These parameters are used to update the aircraft pose in the 3D environment in real time.

The use of WebSocket communication ensures low-latency streaming and compatibility with modern browser-based visualization tools such as CesiumJS [16].

6.6 Implementation Validation

The real-time pipeline was validated through continuous simulation and visualization of multiple aircraft configurations. The system demonstrated:

- Stable real-time execution at fixed simulation time steps
- Consistent aircraft motion between Simulink and visualization client
- Low-latency data transmission through UDP and WebSocket bridge
- Accurate orientation mapping between simulation and visualization

Additionally, the system was tested with different aircraft configurations without requiring modifications to the visualization pipeline. This further confirms that the architecture is independent of aircraft type.

6.7 Summary

The real-time simulation implementation demonstrates that the proposed framework extends beyond analysis into interactive simulation.

The key contribution is the decoupling of physics and visualization, enabling a flexible and extensible architecture that supports real-time simulation, visualization, and pilot interaction.

By integrating a generalized aircraft model with Simulink, UDP communication, and a platform-independent visualization bridge, the framework provides a complete pipeline from physics-based modeling to real-time immersive simulation.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This work presented the development of a modular, aircraft-agnostic simulation framework capable of modeling, analyzing, and simulating a wide range of aircraft configurations within a unified architecture.

The primary contribution of this work is the formulation of a generalized modeling approach in which a single `Aircraft` object serves as the central source of truth for geometry, mass properties, aerodynamics, propulsion, and control systems. By structuring the framework around object-oriented principles and a force and moment aggregation methodology, the need for configuration-specific equations is eliminated. This enables consistent modeling across fundamentally different aircraft types while preserving physical accuracy.

The framework introduces a discipline-based modeling strategy, where each subsystem independently computes its contribution to the total forces and moments. Aerodynamic forces are computed using coefficient-based or lookup-based models (e.g., DATCOM [30]), propulsion forces are generated through modular propulsive elements, and gravitational effects are applied based on mass properties. These contributions are transformed into a common body-axis frame and aggregated about a consistent reference point, ensuring mathematical consistency, modularity, and scalability [1, 2].

A key advantage of this approach is the ability to use a single aircraft model across multiple analysis domains without duplication or redefinition. The framework supports:

- **Trim analysis:** Nonlinear optimization is used to compute steady-state equilibrium conditions across different flight regimes, ensuring force and moment balance.
- **Stability and linearization:** The system is linearized about trim conditions to obtain

state-space models, enabling identification of classical dynamic modes such as short-period, phugoid, Dutch roll, roll subsidence, and spiral modes [3, 4].

- **Performance analysis:** The framework computes key metrics such as rate of climb, takeoff distance, landing distance, and energy-based performance characteristics.
- **Mission simulation:** A mission planner generates full flight trajectories by sequencing trim solutions across altitude and Mach schedules, enabling continuous flight profile evaluation.
- **Time-domain simulation:** Integration with Simulink enables nonlinear 6-DOF simulation using the same aircraft model, ensuring consistency between analysis and simulation environments.

The framework was successfully applied to multiple aircraft configurations, demonstrating both generality and robustness:

- A **Cessna 172**, used for validation against known performance data, including trim conditions, fuel consumption, and brake horsepower.
- An **F-16**, used to demonstrate high-performance flight regimes, including transonic and supersonic conditions, as well as dynamic stability behavior.
- A **Boeing 737** configuration based on DATCOM data [30], used to validate aerodynamic lookup integration and coefficient interpolation across operating conditions.
- A **quadrotor system**, demonstrating the framework’s ability to model rotor-based vehicles where both lift and control moments are generated through propulsion.

Validation results indicate that the framework produces physically consistent outputs across all configurations. Trim solutions converge with negligible force and moment residuals, stability modes align with classical aircraft dynamics theory, and performance metrics fall within reasonable agreement with reference data. Observed behaviors such as spiral mode divergence and phugoid oscillations are consistent with established flight dynamics literature [10].

An additional contribution of this work is the integration of real-time simulation capability. The framework was coupled with Simulink and external visualization systems using UDP and WebSocket communication, enabling real-time state streaming and visualization. This allows the same aircraft model to be used in software-in-the-loop (SIL), hardware-in-the-loop (HIL), and pilot-in-the-loop environments, extending the framework beyond offline analysis into interactive simulation [1, 11].

Overall, this work establishes a unified and extensible foundation for aircraft simulation that bridges conceptual design, flight dynamics analysis, and real-time simulation platforms.

7.1.1 Limitations of This Work

While the proposed framework provides a unified and modular approach to aircraft modeling and simulation, several limitations should be acknowledged.

Aerodynamic model fidelity. The framework primarily utilizes low- to medium-fidelity aerodynamic representations, including coefficient-based models and DATCOM [30]-derived data. These models are suitable for conceptual design and performance estimation, but do not capture high-fidelity effects such as flow separation, unsteady aerodynamics, or aeroelastic coupling. As a result, prediction accuracy may reduce in regimes involving high angles of attack or complex flow behavior.

Simplified geometric representation. Aircraft geometry is represented using reference parameters and component-level abstractions rather than detailed surface discretization. While this is sufficient for force and moment estimation, it does not directly account for geometric interactions such as wing-body interference or local flow variations.

Current propulsion model fidelity. The framework models propulsion at the component level using force and moment contributions, enabling accurate representation of distributed and asymmetric thrust effects. The propulsion system is implemented through an abstract interface, allowing different propulsion models to be integrated without modifying the aircraft-level formulation. In the current implementation, simplified propulsion models are used for performance analysis. However, the framework is not limited to these representations and can incorporate higher-fidelity propulsion models as needed.

Atmospheric model assumptions. The framework currently employs a standard atmosphere (ISA) model to compute air properties. This approach does not account for atmospheric disturbances such as wind, turbulence, or temperature variations, and therefore simplifies environmental effects on flight dynamics.

Absence of sensor and control system dynamics. The present work focuses on aircraft dynamics and force-moment modeling, and does not explicitly include sensor models or feedback control system dynamics. Control inputs are applied directly rather than being

generated through closed-loop control laws. The modular structure allows integration of these components, but they are not included in the current implementation.

Simplified ground interaction modeling. Ground contact is modeled using a basic spring-damper formulation. While this captures general takeoff and landing behavior, it does not represent detailed landing gear mechanics, tire dynamics, or runway interaction effects.

Aeroelastic and structural effects. The aircraft is modeled as a rigid body. Structural flexibility, aeroelastic deformation, and coupling between structural and aerodynamic dynamics are not included in the current formulation.

Numerical dependency in analysis routines. Trim and stability analysis are based on numerical optimization and linearization based on finite-differences. While this provides general applicability across configurations, results may be sensitive to solver settings, convergence tolerances, and numerical step sizes.

These limitations reflect the current scope of the framework, which prioritizes modularity, extensibility, and a unified modeling approach. The architecture is designed such that higher-fidelity models and additional subsystems can be incorporated without modifying the core force and moment formulation.

7.2 Future Work

While the proposed framework provides a strong foundation for generalized aircraft simulation, several extensions can further enhance its capability, fidelity, and applicability.

7.2.1 Extension to eVTOL and Distributed Propulsion Systems

Future work includes extending the framework to electric vertical takeoff and landing (eVTOL) aircraft and distributed propulsion configurations. These systems involve a strong coupling between aerodynamics and propulsion, particularly during the transition between hovering and forward flight. Modeling such interactions accurately requires dynamic blending between aerodynamic and propulsive force contributions, making them an ideal application for the modular architecture proposed in this work [17].

7.2.2 Validation for Conceptual Aircraft Design

The framework can be further developed into a tool for early-stage aircraft design and trade studies. By integrating parametric geometry definitions, weight estimation methods, and performance constraints, it can be used for rapid evaluation of conceptual configurations and design space exploration.

7.2.3 Integration of High-Fidelity Aerodynamics

Currently, aerodynamic models rely on empirical methods such as DATCOM [30] and lookup tables. Future work can include integration with higher-fidelity aerodynamic tools such as computational fluid dynamics (CFD), vortex lattice methods, or panel methods. This would improve predictive accuracy, particularly in nonlinear regimes such as high angles of attack, flow separation, and transonic flight.

7.2.4 Enhanced Visualization and Immersive Simulation

The visualization framework can be expanded to include advanced rendering engines and virtual reality environments. Integration with platforms such as Cesium enables real-time geospatial visualization, but future work may extend this toward immersive pilot-in-the-loop simulations with full cockpit interfaces and sensor visualization.

7.2.5 Generalized Flight Control Framework

A significant extension of this work is the development of a generalized flight control framework compatible with the modular aircraft architecture. This includes implementation of classical and modern control strategies such as PID, linear quadratic regulators (LQR), nonlinear dynamic inversion (NDI), and incremental nonlinear dynamic inversion (INDI).

A unified control framework would enable rapid prototyping and evaluation of control laws across different aircraft configurations without requiring redesign for each platform, which is particularly important for emerging systems such as eVTOL aircraft.

7.2.6 Multi-Physics and System-Level Integration

Future extensions may include integration of additional physical domains such as aeroelasticity, thermal systems, and energy systems. Coupling these effects with flight dynamics would enable comprehensive system-level simulations, particularly for high-performance or long-duration missions.

7.3 Final Remarks

The proposed framework demonstrates that a unified, modular approach to aircraft modeling can significantly improve flexibility, reusability, and scalability in flight simulation. By decoupling subsystem modeling and leveraging a consistent force and moment interface, the framework provides a powerful platform for both engineering analysis and real-time simulation.

This work lays the foundation for future developments in aircraft simulation, particularly in emerging areas such as eVTOL systems, autonomous flight, distributed propulsion, and integrated multi-physics modeling. The ability to represent diverse aircraft configurations within a single framework represents a meaningful step toward more generalized and adaptable aerospace simulation tools.

Bibliography

- [1] B. L. Stevens and F. L. Lewis, *Aircraft Control and Simulation*, 2nd ed. New York, NY, USA: Wiley, 2003.
- [2] B. Etkin and L. D. Reid, *Dynamics of Flight: Stability and Control*, 3rd ed. New York, NY, USA: Wiley, 1996.
- [3] R. C. Nelson, *Flight Stability and Automatic Control*, 2nd ed. New York, NY, USA: McGraw-Hill, 1998.
- [4] M. V. Cook, *Flight Dynamics Principles*, 2nd ed. Oxford, U.K.: Butterworth-Heinemann, 2007.
- [5] U.S. Air Force, “F-16 Fighting Falcon Fact Sheet,” [Online]. Available: <https://www.af.mil/About-Us/Fact-Sheets/Display/Article/104505/> [Accessed: May 5, 2026].
- [6] L. T. Nguyen, M. E. Ogburn, W. P. Gilbert, K. S. Kibler, P. W. Brown, and P. L. Deal, “Simulator Study of Stall/Post-Stall Characteristics of a Fighter Airplane with Relaxed Longitudinal Static Stability,” NASA Technical Paper TP-1538, Dec. 1979.
- [7] Lockheed Martin, “F-16 Fighting Falcon,” [Online]. Available: <https://www.lockheedmartin.com/en-us/products/f-16.html> [Accessed: May 5, 2026].
- [8] SKYbrary Aviation Safety, “General Dynamics F-16 Fighting Falcon,” [Online]. Available: <https://skybrary.aero/aircraft/f16> [Accessed: May 5, 2026].
- [9] U.S. Air Force, *USAF Stability and Control DATCOM*, Wright-Patterson AFB, OH, USA, 1978.
- [10] S. Ngwenya and B. Osano, “Aircraft Stability Characteristics Analysis,” *International Journal of Aerospace Engineering*, 2019.

- [11] J. Berndt, “JSBSim: An Open Source Flight Dynamics Model,” *AIAA Modeling and Simulation Technologies Conference*, 2004.
- [12] FlightGear, “FlightGear Flight Simulator,” 2024. [Online]. Available: <https://www.flightgear.org/>
- [13] Auburn University, “Multi-Aircraft Dynamics, Control, and Simulation Platform,” 2024.
- [14] Laminar Research, “X-Plane Flight Simulator,” 2024. [Online]. Available: <https://www.x-plane.com/>
- [15] MathWorks, “Simulink User’s Guide,” Natick, MA, USA, 2024.
- [16] CesiumGS, “CesiumJS Documentation,” 2024. [Online]. Available: <https://cesium.com/>
- [17] W. Johnson, “Electric VTOL Aircraft Technology Overview,” NASA, 2020.
- [18] G. H. Tomcat, “OpenSim: Browser-Based Flight Simulation Framework,” GitHub Repository, 2024. [Online]. Available: <https://github.com/ghtomcat/opensim>
- [19] AeroSim Open, “AeroSim: Scalable Flight Simulation Framework,” GitHub Repository, 2024. [Online]. Available: <https://github.com/aerosim-open/aerosim>
- [20] AeroSim Documentation, “AeroSim Overview and Architecture,” 2024. [Online]. Available: <https://aerosim.readthedocs.io/>
- [21] M. Colley, J. Czymmek, P. Jansen, L.-M. Meinhardt, P. Ebel, and E. Rukzio, “UAM-SUMO: Simulacra of Urban Air Mobility Using SUMO To Study Large-Scale Effects,” in *Proc. ACM/IEEE Int. Conf. Human-Robot Interaction (HRI)*, Melbourne, Australia, 2025, pp. 1000–1004.
- [22] D. P. Raymer, *Aircraft Design: A Conceptual Approach*, 6th ed. Reston, VA, USA: AIAA, 2018.
- [23] R. W. Beard and T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*. Princeton, NJ, USA: Princeton University Press, 2012.
- [24] J. G. Leishman, *Principles of Helicopter Aerodynamics*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2006.

- [25] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.
- [26] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [27] G. D. Padfield, *Helicopter Flight Dynamics*, 2nd ed. Oxford, U.K.: Blackwell, 2007.
- [28] D. P. Raymer, *Aircraft Design: A Conceptual Approach*, 6th ed. Reston, VA, USA: American Institute of Aeronautics and Astronautics (AIAA), 2018.
- [29] Air Plains Services, “Cessna 172 Performance Data (180 HP Conversion),” [Online]. Available: <https://www.airplains.com/cessna172-180hp> [Accessed: May 5, 2026].
- [30] U.S. Air Force, *USAF Stability and Control DATCOM*, Wright-Patterson AFB, OH, USA, 1978.